

IV. Minimalizacja funkcji metodą tablic Karnaugh'a

Metoda polega na:

wpisaniu funkcji w postaci **ZNPS** do odpowiednio skonstruowanej tablicy, w której łatwo **rozpoznać grupy jedynek** dające się zredukować do **implikantów prostych** oraz bardzo łatwo te **implikanty** wypisać.

lub

wpisaniu funkcji w postaci **ZNPI** do odpowiednio skonstruowanej tablicy, w której łatwo **rozpoznać grupy zer** dające się zredukować do **implicentów prostych** oraz bardzo łatwo te **implicenty** wypisać.

Def:

Implikant funkcji F to taka funkcja **g** (iloczyn argumentów), że jeżeli **g = 1** to funkcja **F = 1** (mówi się, że implikant pokrywa część {lub wszystkie} jedynek funkcji).

Przykład -> $F(DCBA) = \bar{D} \cdot C + C \cdot \bar{B} \cdot A + D \cdot C \cdot B \cdot \bar{A}$ <- Jest to funkcja NPS (suma niepełnych iloczynów).

Można ją zapisać w **liczbowej NPS** -> $F(DCBA) = \Sigma(01XX, X101, 1110)_2$

implikant $g_1 = \bar{D} \cdot C$ pokrywa **4 jedynek** funkcji **F** dla argumentów $(DCBA) = \{0100, 0101, 0110, 0111\}$

implikant $g_3 = D \cdot C \cdot B \cdot \bar{A}$ pokrywa **1 jedynkę** funkcji dla argumentów $(DCBA) = \{1110\}$

Implikant prosty funkcji F to taka funkcja **g** (iloczyn argumentów), która jest **implikantem** i która **zmniejszona o dowolny argument przestaje być implikantem**.

Przykład -> Jeżeli funkcja $F(DCBA) = \bar{D} \cdot C + C \cdot \bar{B} \cdot A + D \cdot C \cdot B \cdot \bar{A}$ jest minimalna (po procesie minimalizacji) to implikant $g_1 = \bar{D} \cdot C$ jest implikantem prostym (szczegóły poniżej). Usunięcie z g_1 np. zmiennej $\bar{D}$ spowoduje, że funkcja przyjmie postać $g_1 = C$ i będzie pokrywać **8 jedynek** dla argumentów $(DCBA) = \{0100, 0101, 0110, 0111, 1100, 1101, 1110, 1111\}$ czyli będzie przyjmować wartość jeden dla takich argumentów, dla których funkcja **F** jest równa **0** (np. dla $(DCBA) = \{1111\}$). Po takim zabiegu funkcja g_1 **przestanie być implikantem** funkcji **F**.

Implicent funkcji F to taka funkcja **h** (suma argumentów), że jeżeli **h = 0** to funkcja **F = 0** (mówi się, że implicent pokrywa część {lub wszystkie} zera funkcji).

Przykład -> $F(DCBA) = (\bar{D}+C) \cdot (C+\bar{B}+A) \cdot (D+C+B+\bar{A})$ <- Jest to funkcja NPI (iloczyn niepełnych sum).

Można ją zapisać w **liczbowej NPI** -> $F(DCBA) = \Pi(10XX, X010, 0001)_2$

implicent $h_1 = (\bar{D}+C)$ pokrywa **4 zera** funkcji **F** dla argumentów $(DCBA) = \{1000, 1001, 1010, 1011\}$

implicent $h_3 = (D+C+B+\bar{A})$ pokrywa **1 zero** funkcji dla argumentów $(DCBA) = \{0001\}$.

Implicent prosty funkcji F to taka funkcja **h** (suma argumentów), która jest **implicentem** i która **zmniejszona o dowolny argument przestaje być implicentem**.

Przykład -> Jeżeli funkcja $F(DCBA) = (\bar{D}+C) \cdot (C+\bar{B}+A) \cdot (D+C+B+\bar{A})$ jest minimalna (po procesie minimalizacji) to implicent $h_1 = (\bar{D}+C)$ jest implicentem prostym (szczegóły poniżej). Usunięcie z h_1 np. zmiennej $\bar{D}$ spowoduje, że funkcja przyjmie postać $h_1 = (C)$ i będzie pokrywać **8 zer** dla argumentów $(DCBA) = \{0000, 0001, 0010, 0011, 1000, 1001, 1010, 1011\}$ czyli będzie przyjmować wartość zero dla takich argumentów, dla których funkcja **F** jest równa **1** (np. dla $(DCBA) = \{0000\}$). Po takim zabiegu funkcja h_1 **przestanie być implicentem** funkcji **F**.

Def:

- Każda funkcje logiczną **F** można przedstawić w postaci sumy wszystkich prostych implikantów

pokrywających wszystkie jedynek funkcji (suma ta ma postać NPS) $F = \sum_{i=1}^n g_i$.

Uwaga: Jeśli funkcja jest w postaci **ZNPS** (suma pełnych iloczynów), to **każdy iloczyn pokrywa tylko 1 jedynkę** funkcji.

Przykład -> $F(DCBA) = \overline{D} \cdot C \cdot \overline{B} \cdot A + D \cdot C \cdot B \cdot \overline{A} \rightarrow$ implikant $g_1 = \overline{D} \cdot C \cdot \overline{B} \cdot A$ pokrywa 1 jedynek funkcji F dla argumentów $(DCBA) = \{0101\}$

- Każda funkcje logiczną F można przedstawić w postaci iloczynu wszystkich prostych implimentów
pokrywających wszystkie zera funkcji (iloczyn ta ma postać NPI) $F = \prod_{i=1}^n h_i$.

Uwaga: Jeśli funkcja jest w postaci ZNPI (iloczyn pełnych sum), to każda suma pokrywa tylko 1 zero funkcji.
Przykład -> $F(DCBA) = (\overline{D} + C + \overline{B} + A) \cdot (D + C + B + \overline{A}) \rightarrow$ implicant $h_1 = (\overline{D} + C + \overline{B} + A)$ pokrywa 1 zero funkcji dla argumentów $(DCBA) = \{1010\}$.

Przykład minimalizacji funkcji nr IV.1

Dokonać minimalizacji funkcji logicznej względem zer i jedynek z wykorzystaniem tablic Karnaugh.
Funkcja podana jest w postaci liczbowej ZNPS : $F(DCBA) = \Sigma(0, 1, 4, 5, 7, 14, 15)$.

Rozwiązanie:

Minimalizacja względem jedynek (ZNPS) – Zapisujemy liczbową ZNPS w postaci binarnej

$$F(DCBA) = \Sigma(0, 1, 4, 5, 7, 14, 15) = \Sigma(\mathbf{0000}, \mathbf{0001}, \mathbf{0100}, \mathbf{0101}, \mathbf{0111}, \mathbf{1110}, \mathbf{1111})_2$$

Konstruujemy tablice Karnaugh'a o kształcie kwadratowym (2x2).

(Uwaga! W przypadku 3 lub 5 argumentów byłby kształt „prawie kwadratowy”, to znaczy odpowiednio 1x2 albo 2x1 lub 2x3 albo 3x2).

Boki tablicy opisujemy w kodzie Graya.

Wiersze tablicy będą odpowiadać argumentom DC a **kolumny** argumentom BA.

Wpisujemy do tablicy funkcję F(DCBA) w ten sposób, że w kratce, której współrzędne określa dana liczba binarna wstawiamy 1.

DC \ BA	00	01	11	10
00	1	1		
01	1	1	1	
11			1	1
10				

Dzięki **opisowi boków** tablicy w **kodzie Graya** poszczególne iloczyny argumentów potencjalnie podlegające sklejaniu – których kody (liczby binarne) różnią się na jednej pozycji binarnej będą położone obok siebie.

Przykładowo w omawianym rozwiązaniu iloczyny, których tło w tablicy zaznaczono kolorem żółtym to:

$$\{\mathbf{0000}, \mathbf{0001}, \mathbf{0100}, \mathbf{0101}\} \Rightarrow \overline{D} \cdot \overline{C} \cdot \overline{B} \cdot \overline{A} + \overline{D} \cdot \overline{C} \cdot \overline{B} \cdot A + \overline{D} \cdot C \cdot \overline{B} \cdot \overline{A} + \overline{D} \cdot C \cdot \overline{B} \cdot A$$

Rozpatrując możliwości sklejeń w poziomych parach iloczynów otrzymamy:

$$\overline{D} \cdot \overline{C} \cdot \overline{B} \cdot \overline{A} + \overline{D} \cdot \overline{C} \cdot \overline{B} \cdot A \Rightarrow \overline{D} \cdot \overline{C} \cdot \overline{B}$$

$$\overline{D} \cdot C \cdot \overline{B} \cdot \overline{A} + \overline{D} \cdot C \cdot \overline{B} \cdot A \Rightarrow \overline{D} \cdot C \cdot \overline{B}$$

$$\text{Co prowadzi ostatecznie do rezultatu } \overline{D} \cdot \overline{C} \cdot \overline{B} + \overline{D} \cdot C \cdot \overline{B} \Rightarrow \overline{D} \cdot \overline{B}$$

To samo uzyskamy stosując drugie zestawienie w pary iloczynów (pary w kolumnach):

$$\overline{D} \cdot \overline{C} \cdot \overline{B} \cdot \overline{A} + \overline{D} \cdot \overline{C} \cdot \overline{B} \cdot A \Rightarrow \overline{D} \cdot \overline{B} \cdot \overline{A}$$

$$\overline{D} \cdot C \cdot \overline{B} \cdot \overline{A} + \overline{D} \cdot C \cdot \overline{B} \cdot A \Rightarrow \overline{D} \cdot \overline{B} \cdot A$$

$$\overline{D} \cdot \overline{B} \cdot \overline{A} + \overline{D} \cdot \overline{B} \cdot A \Rightarrow \overline{D} \cdot \overline{B}$$

Powyższy rezultat uzyskamy tworząc z sąsiadujących jedynek grupy i **pozostawiając jako końcowy efekt sklejania tylko te argumenty, które nie ulegają zmianom we wszystkich sklejanych iloczynach.**

W omawianym przykładzie $\{00\} \Rightarrow \overline{D} \cdot \overline{B}$ – jest to **pierwszy implikant prosty.**

Rozmiary tworzonych grup to kolejne potęgi liczby 2, a więc: 1x2, 2x1, 2x2, 2x4, 4x2, 4x4, 2x8, 8x2, 4x8, 8x4 itd.

DC \ BA	00	01	11	10
00	1	1		
01	1	1	1	
11			1	1
10				

Podobnie postępujemy z pozostałymi grupami jedynek, przy czym może być kilka równoważnych rozwiązań minimalnych, gdyż tworzone grupy mogą (a czasami wręcz powinny się zazębiać – przy projektowaniu układów sekwencyjnych asynchronicznych). **Drugi implikant prosty** (kolor zielony) to **{111}** => **C·B·A**.

Ostatecznie **postać minimalna funkcji F(DCBA)** (jest to NPS) składa się z trzech implikantów prostych i wyraża się zależnością: **F(DCBA) = $\overline{D} \cdot \overline{B} + C \cdot B \cdot A + D \cdot C \cdot B$** .

Uwaga! Tworzymy taką liczbę grup o największych możliwych rozmiarach, tak aby pokryć wszystkie 1 funkcji. Nie używamy pozostałych potencjalnie możliwych do utworzenia grup, gdy dotychczas utworzonymi grupami pokryliśmy już wszystkie jedynki funkcji.

Drugi równoważny wariant rozwiązania otrzymamy tworząc **drugą poziomą grupę dwóch jedynek zazębiającą się z grupą 4 jedynek** (wariant ten również składa się z trzech implikantów prostych).

F(DCBA) = $\overline{D} \cdot \overline{B} + \overline{D} \cdot C \cdot A + D \cdot C \cdot B$ (do sprawdzenia!).

DC \ BA	00	01	11	10
00	1	1		
01	1	1	1	
11			1	1
10				

Minimalizacja względem zer (ZNPI) - Ponieważ funkcja F jest w pełni określona, liczbowa ZNPI uzyskamy z liczbowej ZNPS jako logiczne dopełnienie zbioru jej argumentów:

ZNPS: **F(DCBA) = $\Sigma(0, 1, 4, 5, 7, 14, 15)$** => **ZNPI:** **F(DCBA) = $\Pi(2, 3, 6, 8, 9, 10, 11, 12, 13)$**

Następnie zapisujemy liczbową ZNPI w postaci binarnej

F(DCBA) = $\Pi(2, 3, 6, 8, 9, 10, 11, 12, 13)$ = $\Pi(0010, 0011, 0110, 1000, 1001, 1010, 1011, 1100, 1101)_2$

Konstruujemy **tablice Karnaugh'a o kształcie kwadratowym (2x2)**.

Wpisujemy do tablicy funkcję **F(DCBA)** w ten sposób, że w kratce, której współrzędne określa dana liczba binarna **wstawiamy 0**. Należy zwrócić uwagę, że zera występują w kratkach, w których przy syntezie z wykorzystaniem ZNPS nie występowały 1.

DC \ BA	00	01	11	10
00	1	1	0	0
01	1	1	1	0
11	0	0	1	1
10	0	0	0	0

Korzystając z podanych powyżej reguł dla 1 tworzymy grupy tym razem dla **zer**. Porównując pierwsze i ostatnie liczby opisujące boki tablicy Karnaugh'a (które są w kodzie Gray'a) można zauważyć, że różnią się one również na jednej pozycji kodowej, czyli „sąsiadują” ze sobą. Pozwala to utworzyć grupy 0 (to samo obowiązuje dla 1) tak jak zaznaczono to kolorem czerwonym.

Ostatecznie **postać minimalna funkcji F(DCBA)** (jest to NPI) składa się z trzech implikantów prostych i wyraża się zależnością: **F(DCBA) = $(\overline{D} + B) \cdot (C + B) \cdot (D + \overline{B} + A)$** .

Przykład minimalizacji funkcji nr IV.2

Dokonać minimalizacji funkcji logicznej względem zer i jedynek z wykorzystaniem tablic Karnaugh'a.

Funkcja podana jest w postaci liczbowej ZNPI : **F(xyzw) = $\Pi(0, 2, 8, 10)$** .

Rozwiązanie:

Zapisujemy liczbową ZNPI w postaci binarnej

F(xyzw) = $\Pi(0, 2, 8, 10)$ = $\Pi(0000, 0010, 1000, 1010)_2$

Wpisujemy do tablicy funkcję **F(xyzw)** w ten sposób, że w kratce, której współrzędne określa dana liczba binarna **wstawiamy 0**. Ponieważ funkcja jest w pełni określona w pozostałe kratki wpisujemy 1 (zbiór argumentów, dla którego funkcja przyjmuje wartości 1 jest dopełnieniem zbioru argumentów, dla którego funkcja przyjmuje wartości 0).

xy \ wz	00	01	11	10
00	0	1	1	0
01	1	1	1	1
11	1	1	1	1
10	0	1	1	0

Korzystając z własności liczb opisujących końce tablicy Karnaugh'a (patrz poprzedni przykład) **postać minimalna funkcji $F(xyz)$** wyznaczona w oparciu o **zera** funkcji składa się z **jednego implikentu prostego** i wyraża się zależnością **(jest to NPI)**: $F(xyz) = (y + z)$.

Z kolei minimalizując funkcje względem **1** otrzymujemy 2 grupy **8 jedynek**. Po zastosowaniu wielokrotnych sklejeń otrzymujemy **sumę dwóch implikantów prostych** (jednoelementowych): $F(xyz) = y + z$.

Otrzymana w ten sposób funkcja to NPS (suma jednoczynnikowych iloczynów).

Na tym przykładzie najlepiej widać, że każdy sposób minimalizacji prowadzi do tego samego celu – zawsze można przekształcić funkcję zminimalizowaną względem jedynek w zminimalizowaną względem zer!

Przykład minimalizacji funkcji nr IV.3

Dokonać minimalizacji funkcji logicznej względem zer i jedynek z wykorzystaniem tablic Karnaugh'a.

Funkcja podana jest w postaci liczbowej ZNPS : $F(xyz) = \Sigma (6, 15, (1, 3, 5, 10, 12, 14))$.

Rozwiązanie:

Zapisujemy liczbową ZNPS w postaci binarnej

$$F(xyz) = \Sigma (6, 15, (1, 3, 5, 10, 12, 14)) = \Sigma (0110, 1111, (0001, 0011, 0101, 1010, 1100, 1110))_2$$

Ponieważ funkcja nie jest w pełni określona wpisujemy do tablicy funkcję $F(xyz)$ w ten sposób, że w kratce, której współrzędne określa dana liczba binarna i funkcja jest określona wstawiamy 1 (argumenty {6,15}) a tam gdzie funkcja nie jest określona wstawiamy X (argumenty {1, 3, 5, 10, 12, 14}).

Tworząc grupy jedynek lub zer możemy wykorzystywać wartości X. Dzięki temu mogą powstawać **większe grupy** dające w efekcie prostszą postać funkcji minimalnej. Te same wartości X mogą być wykorzystywane zarówno przy minimalizacji względem jedynek jak i zer.

xy \ wz	00	01	11	10
00	0	X	X	0
01	0	X	0	1
11	X	0	1	X
10	0	0	0	X

Postać minimalna funkcji $F(xyz)$ wyznaczona w oparciu o **jedynki** funkcji składa się z **2 implikantów prostych** i wyraża się zależnością **(jest to NPS)**: $F(xyz) = xyw + ywz$.

Postać minimalna funkcji $F(xyz)$ wyznaczona w oparciu o **zera** funkcji składa się z **3 implikentów prostych** i wyraża się zależnością **(jest to NPI)**: $F(xyz) = (w) \cdot (y) \cdot (x + z)$.

Przykładowe zadania:

Dokonać minimalizacji funkcji logicznej względem zer i jedynek z wykorzystaniem tablic Karnaugh'a:

Funkcje w pełni określone.

a) $F(abcd) = \Sigma (3, 4, 5, 7, 9, 13, 14, 15),$

b) $F(dcba) = \Pi (3, 4, 5, 6, 7, 10, 14),$

Funkcje nie w pełni określone.

c) $F(xyzw) = \Sigma (4, 8, 11, 13, (1, 3, 5, 10, 12, 14)),$

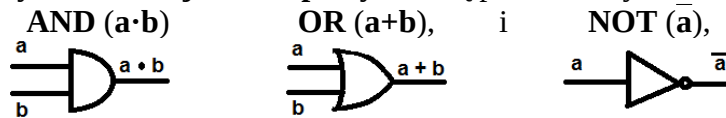
d) $F(abcd) = \Pi (0, 5, 6, 7, 11, 13, (1, 3, 10, 12, 14)).$

V. Synteza strukturalna układów kombinacyjnych

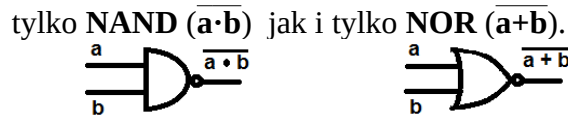
Znając **NPS** lub **NPI** danej funkcji logicznej staramy się za pomocą **funktorów logicznych** stanowiących **system funkcjonalnie pełny** utworzyć układ o zadanym działaniu.

Zbiór funktorów tworzy **system funkcjonalnie pełny** jeżeli za ich pomocą można utworzyć wszystkie funkcje logiczne.

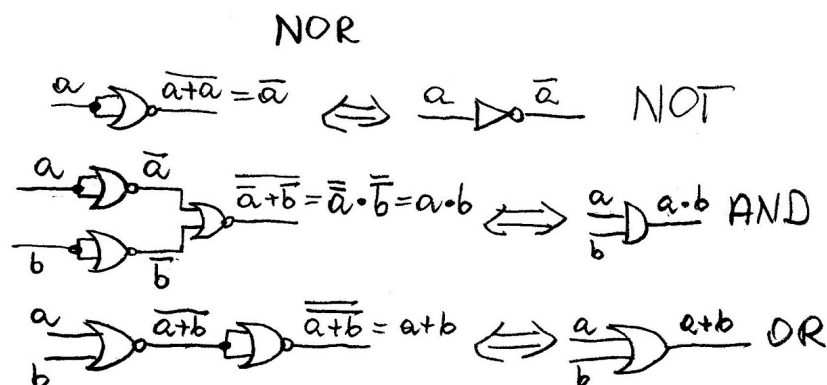
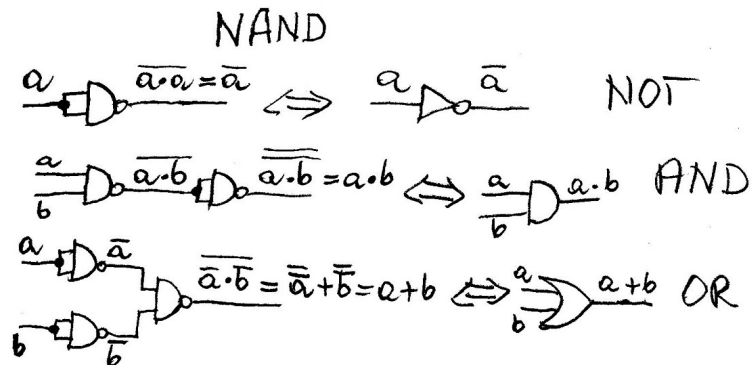
Zbiór funktorów stanowiący **system funkcjonalnie pełny** tworzą przede wszystkim:



ale również



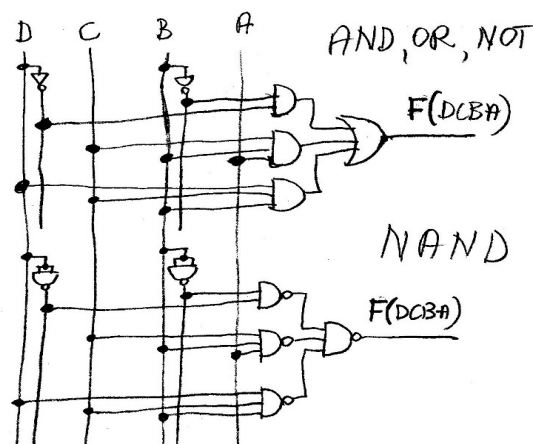
Łatwo wykazać, że stosując aksjomaty i tożsamości algebry Boole'a można wyznaczyć funktory **AND**, **OR** i **NOT** korzystając wyłącznie z samych bramek **NAND** lub samych bramek **NOR**.



Przykład nr V.1

Stosując bramki **AND**, **OR**, **NOT** oraz tylko bramki **NAND** narysować schematy układu kombinacyjnego, którego funkcja **NPS** wyraża się wzorem: **$F(DCBA) = \bar{D} \cdot \bar{B} + C \cdot B \cdot A + D \cdot C \cdot B$**

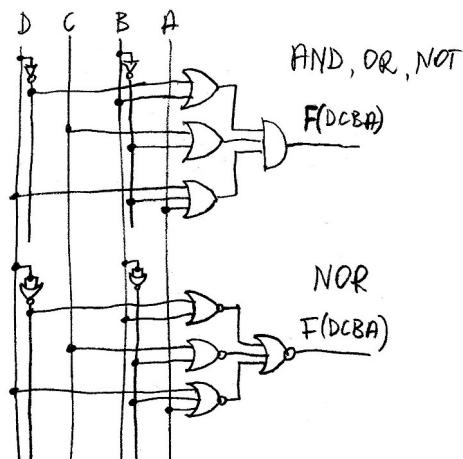
Rozwiązanie: poniżej zamieszczono dwa równoważne schematy. Obydwa wykorzystują pionowe szyny z argumentami funkcji {DCBA}.



Przykład nr V.2

Stosując bramki **AND**, **OR**, **NOT** oraz tylko bramki **NOR** narysować schematy układu kombinacyjnego, którego funkcja **NPI** wyraża się wzorem: $F(DCBA) = (\overline{D} + B) \cdot (C + B) \cdot (D + \overline{B} + A)$

Rozwiązanie: poniżej zamieszczono dwa równoważne schematy. Obydwa wykorzystują pionowe szyny z argumentami funkcji {DCBA}.



UWAGI: Dokonując syntezy NPS lub NPI za pomocą bramek **któregokolwiek systemu funkcjonalnie pełnego otrzymujemy układ trójwarstwowy – optymalny z punktu widzenia opóźnienia wnoszonego przez układ logiczny**. Opóźnienie to jest równe dwóm (2) lub trzem (3) czasom propagacji pojedynczej bramki logicznej (lepiej już nie można!!!).

W przypadku **NPS** mamy:

- **warstwę negatorów** (nie wszystkie są potrzebne),
- **warstwę mnożącą** obliczającą iloczyny logiczne argumentów,
- **warstwę sumującą** obliczającą sumę logiczną wszystkich iloczynów.

W przypadku **NPI** mamy:

- **warstwę negatorów** (nie wszystkie są potrzebne),
- **warstwę sumującą** obliczającą sumy logiczne argumentów,
- **warstwę mnożącą** obliczającą iloczyn logiczny wszystkich sum.

Najczęściej do syntezy stosuje się **system funkcjonalnie pełny oparty jednorodnie bramki albo NAND albo NOR**.

Do syntezy NPS stosujemy bramki NAND!!! (Przykład nr V.1)

Do syntezy NPI stosujemy bramki NOR!!! (Przykład nr V.2)

Zwykle w warstwie negatorów stosuje się zwykłe bramki NOT!!!

Przykładowe zadania.

Zad. V.1

Dana jest funkcja logiczna: $f(abc) = \bar{a} \cdot b + a \cdot (\bar{c} + \bar{b})$.

- Wykorzystując metodę tablicowania funkcji wyznaczyć postać liczbową ZNPS i ZNPI.
- Zminimalizować funkcję względem zer i jedynek.

$$f(abc) = \bar{a}b + a(\bar{c} + \bar{b})$$

Tabela pomocnicza do wyznaczenia ZNPS i ZNPI funkcji $f(abc)$

abc	$\bar{a}b$	$\bar{c} + \bar{b}$	$a(\bar{c} + \bar{b})$	$f(abc)$
000	0	1	0	0
001	0	1	0	0
010	1	1	0	1
011	1	0	0	1
100	0	1	1	1
101	0	1	1	1
110	0	1	1	1
111	0	0	0	0

ZNPS

$$f(abc) = \Sigma(2, 3, 4, 5, 6)$$

ZNPI

$$f(abc) = \Pi(0, 1, 7)$$

Tablica Karnaugh'a $f(abc)$

c	0	1
ab	0 0	0 0
01	1 1	1 1
11	1 0	1 0
10	1 1	1 1

$$f(abc) = (a+b)(\bar{a}+\bar{b}+\bar{c}) = \bar{a}b + b\bar{c} + a\bar{b}$$

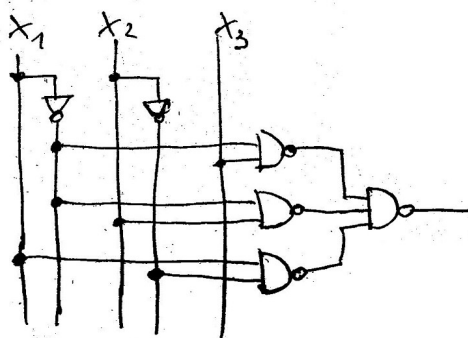
Zad. V.2

Zaprojektować układ kombinacyjny dający na wyjściu 1 logiczną, gdy czterobitowa liczba N, w naturalnym systemie dwójkowym podana na wejście spełnia warunek $2 \leq N \leq 11$. Układ zrealizować przy wykorzystaniu funktorów NAND (jako ćwiczenie wykonać syntezę przy wykorzystaniu funktorów NOR).

x_1	x_2	x_3	x_4	
0000				0
0001				0
0010				1
0011				1
0100				1
0101				1
0110				1
0111				1
1000				1
1001				1
1010				1
1011				1
1100				0
1101				0
1110				0
1111				0

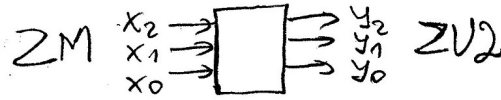
$x_1 x_2$	$x_3 x_4$ 00	01	11	10
00			1	1
01	1	1	1	1
11				
10	1	1	1	1

$$f(x_1, x_2, x_3, x_4) = \bar{x}_1 x_3 + \bar{x}_1 x_2 + x_1 \bar{x}_2$$



Zad. V.3

Zaprojektować układ kombinacyjny zamieniający 3 bitową liczbę ze znakiem w systemie zapisu **znak moduł (ZM)** na **znak uzupełnienie do dwóch (ZU2)**. Układ zrealizować przy wykorzystaniu funkcyjów **NAND**.
(jako ćwiczenie wykonać syntezę przy wykorzystaniu funkcyjów **NOR**).



$x_2 x_1 x_0$	$y_2 y_1 y_0$
0 0 0	0 0 0
0 0 1	0 0 1
0 1 0	0 1 0
0 1 1	0 1 1
1 0 0	X X X
1 0 1	1 1 1
1 1 0	1 1 0
1 1 1	1 0 1

Wartości dokładnie w obydwu systemach są takie same?

W systemie ZM nie występuje -0 // ? stąd w ZU2 wartości nieokreślone

$$\begin{aligned} \text{ZU2: } "1" &\rightarrow 001 & "3" &\rightarrow 011 \\ &+ 110 & &+ 100 \\ &\hline &111 & &101 \\ "1" &\rightarrow 111 & "3" &\rightarrow 101 \\ "2" &\rightarrow 010 & & \\ &+ 101 & & \\ &\hline &111 & & \end{aligned}$$

tablice Karnaugh'a dla y_2

$x_2 \backslash x_1 x_0$	00	01	11	10
00				
01				
11	1	1		
10	X		1	

tablice Karn. dla y_1

$x_2 \backslash x_1 x_0$	00	01	11	10
00				
01				
11	1	1		
10	X		1	

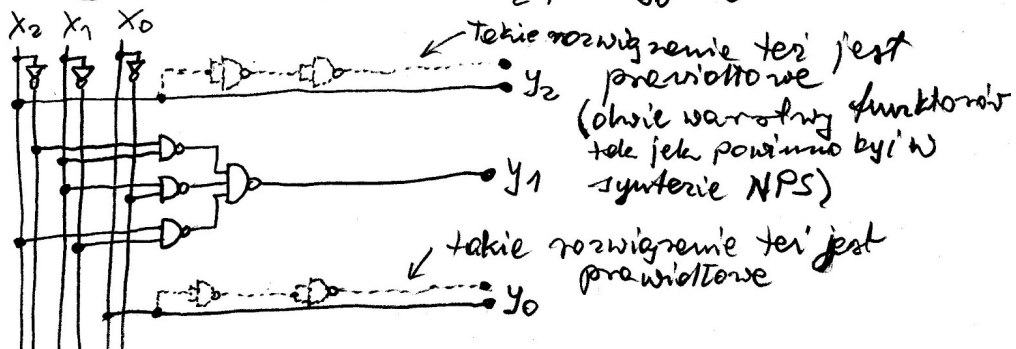
tabl. Karn. dla y_0

$x_2 \backslash x_1 x_0$	00	01	11	10
00				
01				
11	1	1		
10	X		1	

$$y_2 = x_2$$

$$y_1 = \bar{x}_2 x_1 + x_2 \bar{x}_0 + x_2 \bar{x}_1$$

$$y_0 = x_0$$



Zad. V.4

Zaprojektować układ kombinacyjny realizujący następującą funkcję przy wykorzystaniu funktorów **NAND** (jako ćwiczenie wykonać syntezę przy wykorzystaniu funktorów **NOR**).

$$F(S_1, S_2, x_1, x_2, x_3) = \begin{cases} f_1(x_1, x_2, x_3) & \text{dla } S_1=0, S_2=0 \\ f_2(x_1, x_2, x_3) & \text{-- } S_1=0, S_2=1 \\ f_3(x_1, x_2, x_3) & \text{-- } S_1=1, S_2=0 \\ f_4(x_1, x_2, x_3) & \text{-- } S_1=1, S_2=1 \end{cases}$$

$$f_1(x_1, x_2, x_3) = \Sigma(0, 1, 2, 4, 5, 6)$$

$$f_2(x_1, x_2, x_3) = \Pi(1, 3, 5, 7)$$

$$f_3(x_1, x_2, x_3) = \bar{x}_1 + x_2 \bar{x}_3$$

$$f_4(x_1, x_2, x_3) = x_1 x_2 x_3 + \bar{x}_2 \bar{x}_3$$

przekształcamy f_3 i f_4 z NPS do ZNPS

$$\begin{aligned} f_3(x_1, x_2, x_3) &= \bar{x}_1 + \bar{x}_2 \bar{x}_3 \xrightarrow{\text{linbowe NPS}} \Sigma(0xx, x00)_2 = \Sigma(000, 001, 010, 011, 000, 100)_2 \\ &= \Sigma(0, 1, 2, 3, 0, 4) = \Sigma(0, 1, 2, 3, 4) \end{aligned}$$

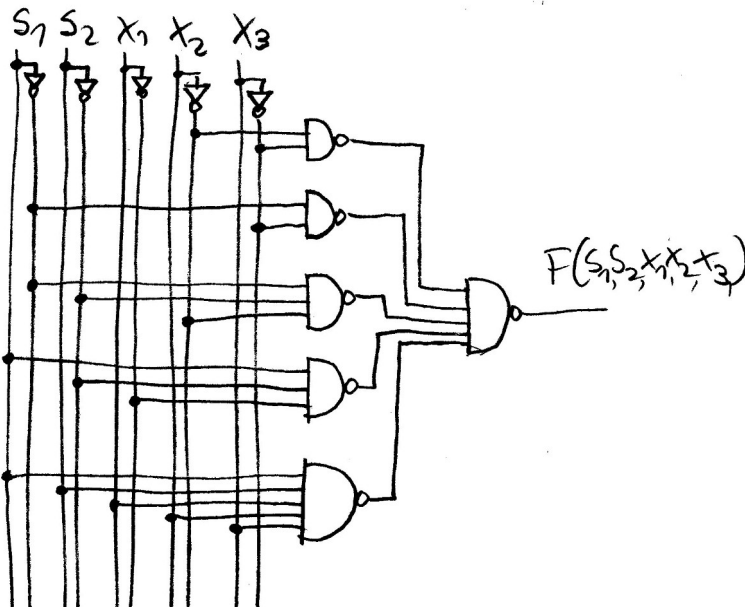
↑
powstałe z 0

$$\begin{aligned} f_4(x_1, x_2, x_3) &= x_1 x_2 x_3 + \bar{x}_2 \bar{x}_3 \xrightarrow{\text{linbowe NPS}} \Sigma(111, x00)_2 = \Sigma(7, 0, 4) \\ &= \Sigma(0, 4, 7) \end{aligned}$$

wpisujemy funkcje $F(S_1, S_2, x_1, x_2, x_3)$ do odpowiednio ukształtowanej tablicy karnaugh'a

$x_1 x_2 x_3$	000	001	011	010	110	111	101	100
$S_1 S_2$								
00	1	1	1	1	1	1	1	1
01	1	0	0	1	1	0	0	1
11	1	1	1	1	1	1	1	1
10	1	1	1	1	1	1	1	1

$$F(S_1, S_2, x_1, x_2, x_3) = \bar{x}_2 \bar{x}_3 + \bar{x}_1 \bar{x}_3 + \bar{x}_1 \bar{x}_2 + S_1 \bar{x}_2 \bar{x}_1 + S_1 S_2 x_1 x_2 x_3$$

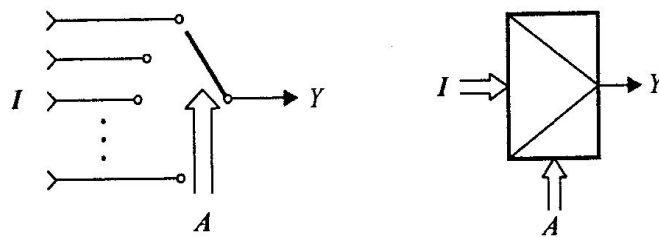


VI. Synteza strukturalna układów kombinacyjnych z wykorzystaniem multiplekserów

Bibliografia:

- M. Barski, W. Jędruch, *Układy cyfrowe, podstawy projektowania i opis w języku VHDL*, Wydawnictwo PG, Gdańsk 2007.
- H. Kruszyński, A. Rydzewski, A. Śluzek, *Teoria układów cyfrowych, materiały pomocnicze do ćwiczeń i laboratorium*, Wydawnictwo PW, Warszawa 1987.

Multiplekser to układ o N wejściach adresowych A i $2^N = Q$ wejściach informacyjnych I i jednym wyjściu Y , łączonym z jednym wejściem w zależności od podanego adresu. Oznacza się go symbolem $M \times Q/1$, wartości Q najczęściej należą do zbioru $Q = \{2, 4, 8, 16\}$.



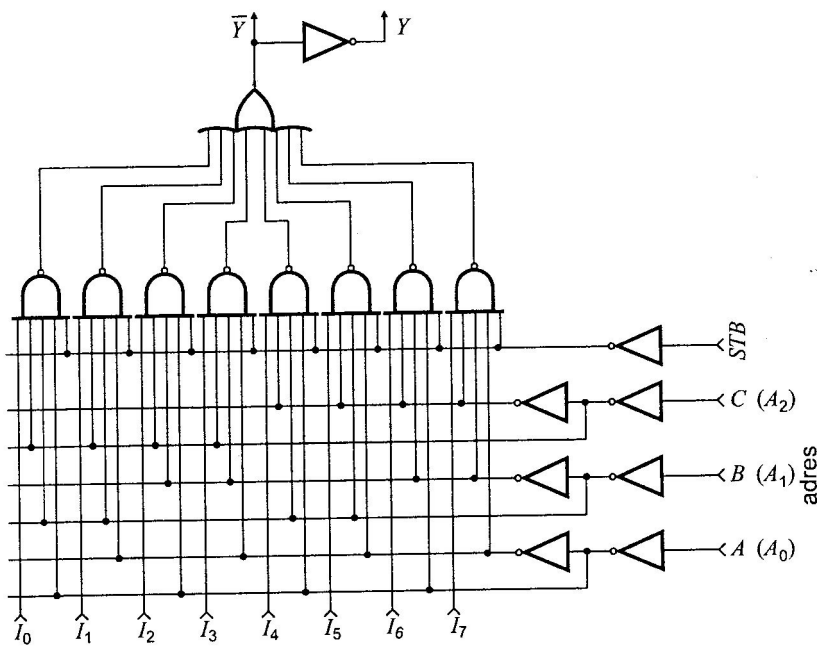
Funkcja opisująca wyjście takiego multipleksera ma postać:

$$Y = I_0 \bar{A}_{N-1} \bar{A}_{N-2} \dots \bar{A}_2 \bar{A}_1 \bar{A}_0 + I_1 \bar{A}_{N-1} \bar{A}_{N-2} \dots \bar{A}_2 \bar{A}_1 A_0 + I_2 \bar{A}_{N-1} \bar{A}_{N-2} \dots \bar{A}_2 A_1 \bar{A}_0 + \dots + I_{Q-1} A_{N-1} A_{N-2} \dots A_2 A_1 A_0$$

Przykładowo dla multipleksera $M \times 8/1$ o $N=3$ wejściach adresowych A funkcja ta jest równa:

$$Y = I_0 \bar{A}_2 \bar{A}_1 \bar{A}_0 + I_1 \bar{A}_2 \bar{A}_1 A_0 + I_2 \bar{A}_2 A_1 \bar{A}_0 + I_3 \bar{A}_2 A_1 A_0 + I_4 A_2 \bar{A}_1 \bar{A}_0 + I_5 A_2 \bar{A}_1 A_0 + I_6 A_2 A_1 \bar{A}_0 + I_7 A_2 A_1 A_0$$

Przykładową realizację $M \times 8/1$, za pomocą układu TTL **74151** przedstawia poniższy schemat (**STB** to wejście strobowe - blokujące).

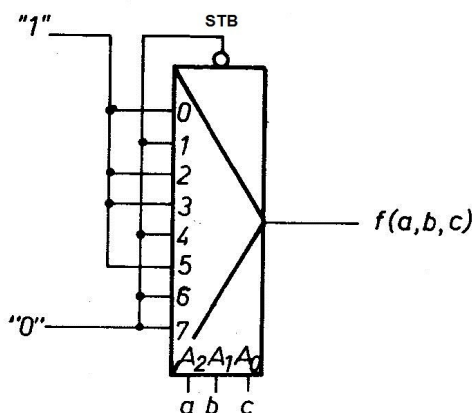


Za pomocą multipleksera o N wejściach adresowych A można zrealizować dowolną funkcję N zmiennych podając jej argumenty na wejścia adresowe A , zaś wejścia informacyjne I łącząc z 0 lub 1 zgodnie z tabelą funkcji.

Przykład nr VI.1

Zrealizować funkcję $f(a,b,c)=\Sigma(0, 2, 3, 5)$ na $Mx 8/1$.

Rozwiązanie: Należy na wejścia adresowe A_2, A_1, A_0 podać odpowiednio argumenty funkcji a, b, c , zaś na wejścia informacyjne o numerach $0, 2, 3, 5$ podać 1 a na pozostałe wejścia 0 (patrz schemat poniżej).

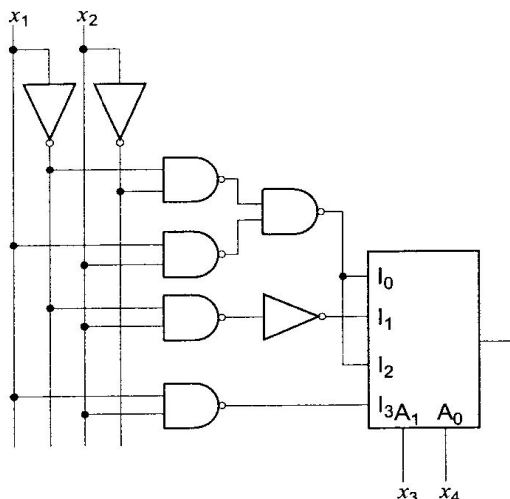


Funkcję o **większej niż N liczbie zmiennych** można zrealizować podając N jej argumentów (wybór jest dowolny, ale najlepiej wybrać N pierwszych lub N ostatnich zmiennych) na wejścia adresowe, zaś $2^N = Q$ odpowiednich funkcji pozostałych argumentów na wejścia informacyjne I .

Przykład nr VI.2

Zrealizować funkcję $f(x_1, x_2, x_3, x_4)=\Sigma(0, 2, 3, 5, 7, 11, 12, 14)$ na $Mx 4/1$ oraz bramkach NAND (i negatorach). Na zmienne adresowe A_1, A_0 podać argumenty x_3, x_4 .

Rozwiązanie: Na na schemacie poniżej zamieszczono zaprojektowany układ kombinacyjny. Jego syntezę należy wykonać jako ćwiczenie po zapoznaniu się z materiałem zamieszczonym w następnym podpunkcie opracowania. Jak można zauważyć wyjścia 4 funkcji zmiennych x_1, x_0 zrealizowane na bramkach NAND i negatorach (synteza NPS) podano na wejścia informacyjne multipleksera.



Przykład nr VI.3

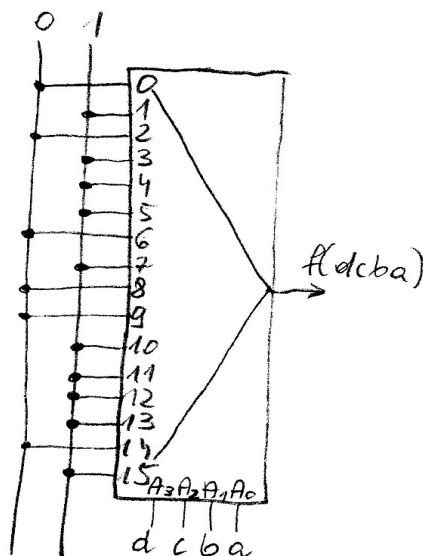
Funkcję $f(d, c, b, a) = d\bar{c}b + ca + c\bar{b} + \bar{d}a$ zrealizować na różne sposoby w oparciu o multipleksery $Mx Q/1$, gdzie $Q=2^N=\{16, 8, 4\}$ ($N=\{4, 3, 2\}$) oraz jeśli to konieczne z wykorzystaniem bramek NAND lub NOR. Jeśli liczba argumentów **będzie mniejsza od liczby zmiennych adresowych** $A_{N-1}, \dots, A_0$ to na zmienne adresowe należy podać argumenty **bardziej znaczące** $\{d, c, b\}$ lub $\{d, c\}$.

Rozwiązanie: Funkcja $f(d, c, b, a) = d\bar{c}b + ca + c\bar{b} + \bar{d}a$ jest w postaci NPS i w pierwszym kroku należy ją zamienić na **liczbową ZNPS**. W tym celu zostanie wykorzystana **liczbowa NPS**:

$$\begin{aligned}
 f(d, c, b, a) &= d\bar{c}b + ca + c\bar{b} + \bar{d}a = \Sigma(101X, X1X1, X10X, 0XX1)_2 \leftarrow \text{liczbowa NPS} \\
 &= \Sigma(1010, 1011, 0101, 0111, 1101, 1111, 0100, 0101, 1100, 1101, 0001, 0011, 0101, 0111)_2 \\
 &= \Sigma(10, 11, 5, 7, 13, 15, 4, 5, 12, 13, 1, 3, 5, 7) \\
 &= \Sigma(1, 3, 4, 5, 7, 10, 11, 12, 13, 15) \leftarrow \text{liczbowa ZNPS}
 \end{aligned}$$

Q = 16 -> synteza układu z wykorzystaniem Mx 16/1

Na wejścia adresowe A_3, A_2, A_1, A_0 należy podać odpowiednio argumenty funkcji d, c, b, a , zaś na wejścia informacyjne o numerach 1, 3, 4, 5, 7, 10, 11, 12, 13, 15 podać 1 a na pozostałe wejścia 0 (patrz schemat poniżej).



Uwaga! Gdyby funkcja $f(d, c, b, a)$ była nie w pełni określona np.:

$$f(d, c, b, a) = \Sigma(1, 3, 4, 5, 7, 10, 11, 12, 13, (0, 15))$$

to na wejścia informacyjne o numerach 0, 15 można by podać zarówno 1 jak i 0 (nie ma to znaczenia).

Q = 8 -> synteza układu z wykorzystaniem Mx 8/1

Liczba argumentów jest większa od liczby wejść adresowych. Zgodnie z podaną sugestią na wejścia adresowe A_2, A_1, A_0 należy podać argumenty funkcji d, c, b , zaś na wejścia informacyjne $Q = 8$ odpowiednich funkcji pozostałego argumentu a . Funkcje te wyznaczymy 2 metodami.

Metoda I

Zamieniamy liczbową ZNPS na postać jawną funkcji czyli ZNPS.

$$\begin{aligned} f(d, c, b, a) &= \Sigma(1, 3, 4, 5, 7, 10, 11, 12, 13, 15) \\ &= \Sigma(0001, 0011, 0100, 0101, 0111, 1010, 1011, 1100, 1101, 1111)_2 \\ &= \bar{d}\bar{c}\bar{b}a + \bar{d}\bar{c}b\bar{a} + \bar{d}c\bar{b}\bar{a} + \bar{d}c\bar{b}a + \bar{d}c b\bar{a} + \bar{d}c b a + d\bar{c}\bar{b}\bar{a} + d\bar{c}\bar{b}a + d\bar{c}b\bar{a} + d\bar{c}b a \\ &\quad + d c\bar{b}\bar{a} + d c\bar{b}a + d c b\bar{a} + d c b a \end{aligned}$$

W poszczególnych iloczynach bierzemy w nawias argument którego funkcje chcemy wyznaczać, w tym przypadku jest to a .

$$f(d, c, b, a)$$

$$= \bar{d}\bar{c}\bar{b}(a) + \bar{d}\bar{c}b(a) + \bar{d}c\bar{b}(a) + \bar{d}c\bar{b}(a) + \bar{d}c b(a) + \bar{d}c\bar{b}(a) + \bar{d}c b(a) + \bar{d}c\bar{b}(a) + \bar{d}c b(a) + \bar{d}c\bar{b}(a) + \bar{d}c b(a) + \bar{d}c\bar{b}(a) + \bar{d}c b(a) + \bar{d}c\bar{b}(a) + \bar{d}c b(a) + \bar{d}c\bar{b}(a)$$

Dokonujemy ewentualnych sklejeń argumentu a .

$$f(d, c, b, a) = \bar{d}\bar{c}\bar{b}(a) + \bar{d}\bar{c}b(a) + \bar{d}c\bar{b}(1) + \bar{d}c\bar{b}(a) + \bar{d}c b(1) + \bar{d}c b(a) + d\bar{c}\bar{b}(1) + d\bar{c}\bar{b}(a) + d\bar{c}b(1) + d\bar{c}b(a) + dc\bar{b}(1) + dc\bar{b}(a) + dc b(1) + dc b(a) + dc\bar{b}(1) + dc\bar{b}(a)$$

Zakodowując argumenty d, c, b otrzymamy adresy wejść informacyjnych, na które należy wysłać zawartości nawiasów czyli funkcje zmiennej a . Jeśli brakuje któregoś z adresów to dopisujemy do funkcji $f(d, c, b, a)$ odpowiadający mu iloczyn argumentów d, c, b mnożąc go przez zero w nawiasie (0). W tym przypadku będzie to (0100) -> $\bar{d}c\bar{b}$.

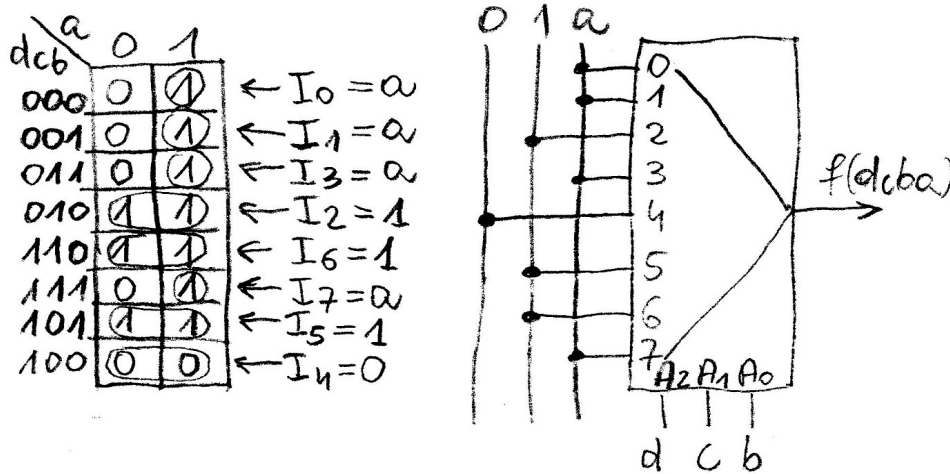
$$\begin{aligned} f(d, c, b, a) &= \bar{d}\bar{c}\bar{b}(a) + \bar{d}\bar{c}b(a) + \bar{d}c\bar{b}(1) + \bar{d}c\bar{b}(a) + \bar{d}c b(1) + \bar{d}c b(a) + d\bar{c}\bar{b}(1) + d\bar{c}\bar{b}(a) + d\bar{c}b(1) + d\bar{c}b(a) + dc\bar{b}(1) + dc\bar{b}(a) + dc b(1) + dc b(a) + dc\bar{b}(1) + dc\bar{b}(a) \\ &\quad (000)(a) \quad (001)(a) \quad (010)(1) \quad (011)(a) \quad (101)(1) \quad (110)(1) \quad (111)(a) \quad (100)(0) \\ &\quad (0)(a) \quad (1)(a) \quad (2)(1) \quad (3)(a) \quad (5)(1) \quad (6)(1) \quad (7)(a) \quad (4)(0) \end{aligned}$$

Na podstawie tak przekształconej funkcji rysujemy schemat układu.

Metoda II (polecana!)

Wpisujemy liczbową ZNPS do odpowiednio ukształtowanej tablicy Karnaugh -> 3x1 tak, aby zmienne **d, c, b** były umieszczone razem na jednym boku. Dla poszczególnych wartości argumentów **d, c, b** odpowiadających zmiennym adresowym multiplexera minimalizujemy funkcje argumentu **a**.

UWAGA! Należy pamiętać, że adresy zapisane są w kolejności odpowiadającej liczbom uszeregowanym w kodzie Gray'a.



Q = 4 -> synteza układu z wykorzystaniem Mx 4/1

Liczba argumentów jest większa od liczby wejść adresowych. Zgodnie z podaną sugestią na wejścia adresowe **A₁, A₀** należy podać argumenty funkcji **d, c**, zaś na wejścia informacyjne **Q = 4** odpowiednie funkcje pozostałych argumentów **b, a**. Funkcje te wyznaczmy również 2 metodami.

Metoda I

Zamieniamy liczbową ZNPS na postać jawną funkcji czyli ZNPS.

$$\begin{aligned} f(d, c, b, a) &= \Sigma(1, 3, 4, 5, 7, 10, 11, 12, 13, 15) \\ &= \Sigma(0001, 0011, 0100, 0101, 0111, 1010, 1011, 1100, 1101, 1111)_2 \\ &= \bar{d}\bar{c}\bar{b}a + \bar{d}\bar{c}b\bar{a} + \bar{d}c\bar{b}\bar{a} + \bar{d}c\bar{b}a + \bar{d}c b\bar{a} + d\bar{c}\bar{b}\bar{a} + d\bar{c}b\bar{a} + d\bar{c}\bar{b}a + d\bar{c}b\bar{a} + d\bar{c}b\bar{a} \end{aligned}$$

W poszczególnych iloczynach bierzemy w nawias argumenty których funkcje chcemy wyznaczać, w tym przypadku są to **b, a**.

$$\begin{aligned} f(d, c, b, a) &= \bar{d}\bar{c}(\bar{b}a) + \bar{d}\bar{c}(b\bar{a}) + \bar{d}c(\bar{b}\bar{a}) + \bar{d}c(\bar{b}a) + \bar{d}c(b\bar{a}) + d\bar{c}(\bar{b}\bar{a}) + d\bar{c}(b\bar{a}) + d\bar{c}(\bar{b}a) + d\bar{c}(b\bar{a}) + d\bar{c}(b\bar{a}) \end{aligned}$$

Dla poszczególnych iloczynów **d, c** porządkujemy wyrażenia w nawiasach (sumujemy w jednym nawiasie).

$$f(d, c, b, a) = \bar{d}\bar{c}(\bar{b}a + b\bar{a}) + \bar{d}c(\bar{b}\bar{a} + \bar{b}a + b\bar{a}) + d\bar{c}(b\bar{a} + b\bar{a}) + d\bar{c}(\bar{b}\bar{a} + \bar{b}a + b\bar{a}).$$

Wyrażenia w nawiasach minimalizujemy korzystając z aksjomatów i tożsamości algebry Boole'a lub za pomocą pomocniczych tablic Karnaugh'a zmiennych **b, a**.

Ostatecznie otrzymujemy:

$$f(d, c, b, a) = \bar{d}\bar{c}(a) + \bar{d}c(\bar{b} + a) + d\bar{c}(b) + d\bar{c}(\bar{b} + a).$$

Zakodowując argumenty **d, c** otrzymamy adresy wejść informacyjnych, na które należy wysłać zawartości nawiasów czyli funkcje zmiennych **b, a**. Jeśli brakuje któregoś z adresów to dopisujemy do funkcji **f(d, c, b, a)** odpowiadający mu iloczyn argumentów **d, c** mnożąc go przez **zero** w nawiasie (0).

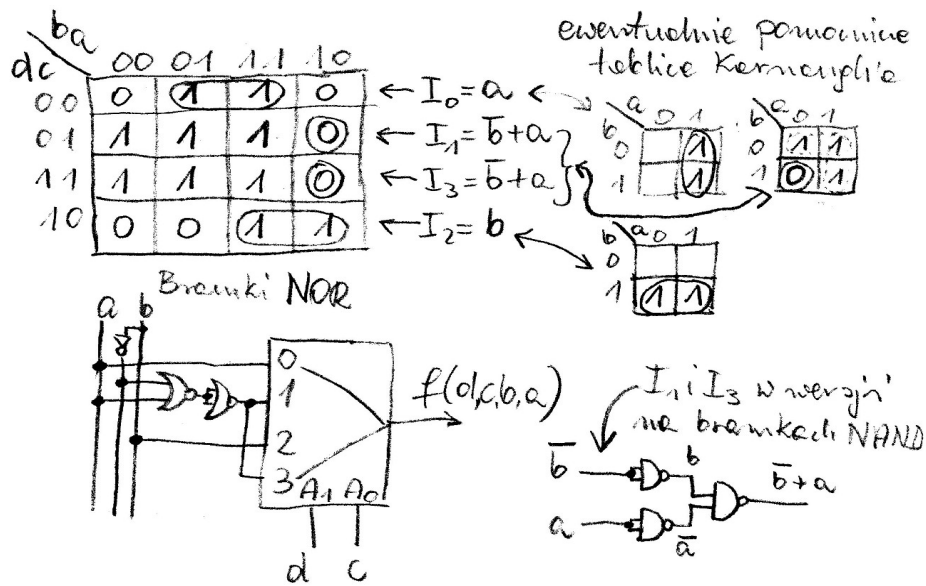
$$\begin{aligned} f(d, c, b, a) &= \bar{d}\bar{c}(a) + \bar{d}c(\bar{b} + a) + d\bar{c}(b) + d\bar{c}(\bar{b} + a) \\ &= (00)(a) + (01)(\bar{b} + a) + (10)(b) + (11)(\bar{b} + a) \\ &= (0)(a) + (1)(\bar{b} + a) + (2)(b) + (3)(\bar{b} + a) \end{aligned}$$

Na podstawie tak przekształconej funkcji rysujemy schemat układu.

Metoda II (polecana!)

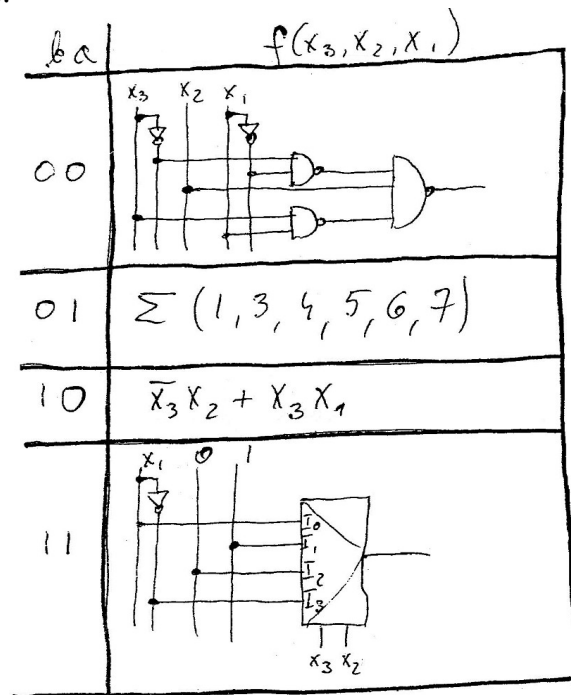
Wpisujemy **liczbową ZNPS** do odpowiednio ukształtowanej **tablicy Karnaugh** -> 2x2 tak, aby zmienne **d, c** były umieszczone razem na jednym boku. Dla poszczególnych wartości argumentów **d, c** odpowiadających zmiennym adresowym multiplexera minimalizujemy funkcje argumentów **b, a**.

UWAGA! Należy pamiętać, że **adresy zapisane są w kolejności odpowiadającej liczbom uszeregowanym w kodzie Gray'a**.



Zad. VI.1

Zaprojektować z wykorzystaniem multiplexera **Mx 8/1** oraz bramek **NAND** układ realizujący następującą funkcję 5-ciu zmiennych **F(b, a, x₃, x₂, x₁)**:



W rozwiązaniu podać:

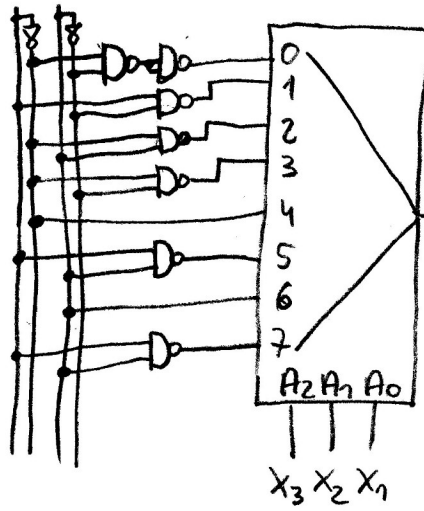
- tablicę Karnaugh'a funkcji **F(b, a, x₃, x₂, x₁)**,
- postać **liczbową ZNPS** tej funkcji,
- minimalne funkcje zmiennych **b, a** podawane na wejścia informacyjne multiplexera,
- schemat układu.

Rozwiązanie: W pierwszej kolejności należy poszczególne funkcje składowe odpowiadające kolejnym wartościom argumentów **b, a** przekształcić do **liczbowych ZNPS**.

- Dla **(b, a) = (00)₂** należy tak przekształcić schemat funkcji **NPS** aby składał się z **3 warstw funkcyjnych NAND**. Dlatego też zmienną **x₂** należy połączyć przez dwa szeregowo połączone negatory i sygnał wejściowy odczytać po pierwszym negatorze – po warstwie negującej.

Schemat układu zamieszczono poniżej.

b a



$F(b, a, x_3, x_2, x_1)$

UWAGA!

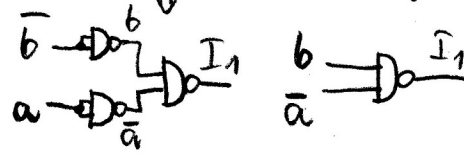
Dla wszystkich sum

zastosowano uproszczenie

np. I_1

powinno być

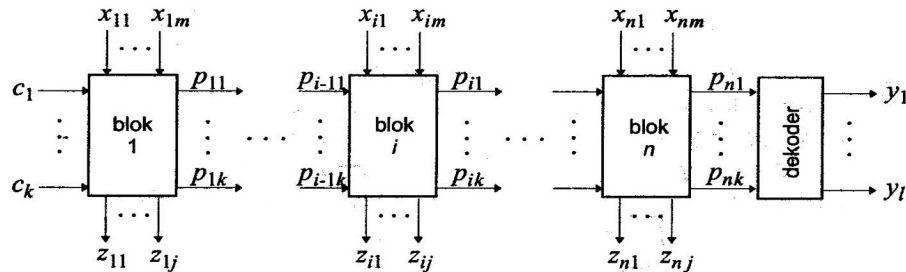
jest.



VII. Projektowanie układów iteracyjnych

Układy iteracyjne wykorzystuje się przy projektowaniu układów kombinacyjnych o wielu wejściach. Warunkiem koniecznym możliwości wykorzystania tej metody projektowej jest możliwość opisu układu w postaci rekurencyjnej, tzn. takiej, że każde kolejne wejście (lub grupa wejść) w identyczny sposób modyfikuje informację o wejściach dotychczas rozpatrywanych. W konsekwencji układ ma postać łańcucha identycznych bloków, zakończonego opcjonalnym blokiem wyjściowym (dekoderem). Informacja o dotychczas przeanalizowanych wejściach, przesyłana do następnego bloku, może być traktowana jako aktualny stan tych wejść.

W iteracyjnych układach kombinacyjnych przez stan układu rozumiemy chwilową wartość przeniesień!!



Etapy projektowania układów iteracyjnych:

- ustalenia liczby wejść x_i podawanych do jednego bloku,
- zdefiniowanie i zakodowanie stanów, co określa liczbę sygnałów przesyłanych pomiędzy blokami,
- konstrukcje tablicy przejść stanów,
- jeśli to konieczne konstrukcja tablicy bloku wyjściowego (dekodera),
- synteza bloku na podstawie tablicy przejść stanów,
- ustalenie stanów początkowych,
- ewentualna synteza bloku wyjściowego (dekodera).

Przykład nr VII.1

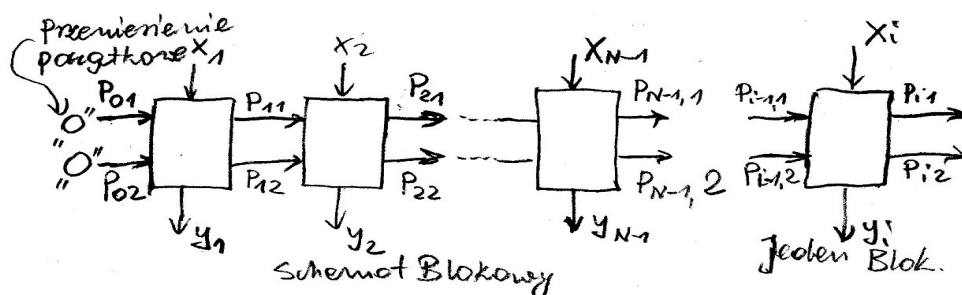
Zaprojektować i zrealizować na bramkach **NAND** iteracyjny układ kombinacyjny analizujący **N** elementową kombinację wejściową i **przepuszczający na wyjście kolejne bity** dopóki w dotychczas przeanalizowanej kombinacji nie występują **więcej niż dwa zera**. Pozostałe bity maskujemy (zastępujemy) zerami.

Rozwiązanie: Projektowany układ będzie zbudowany z jednakowych bloków posiadających po jednym wejściu x_i i jednym wyjściu y_i . Nie będzie potrzebny dekodер, gdyż każdy blok będzie generował bieżący sygnał y_i . Aby narysować schemat blokowy układu oraz schemat jednego bloku iteracyjnego konieczne jest ustalenie liczby stanów a tym samym liczby bitów przeniesienia. Konstruujemy poniższą tabelę:

STAN	KOD	
	p_{i1}	p_{i2}
W dotychczas przeanalizowanej kombinacji nie wystąpiły zera.	0	0
W dotychczas przeanalizowanej kombinacji wystąpiło jedno zero.	0	1
W dotychczas przeanalizowanej kombinacji wystąpiły dwa zera.	1	0
W dotychczas przeanalizowanej kombinacji wystąpiły więcej niż dwa zera.	1	1

W pierwszej kolejności należy określić i wypisać słownie wszystkie spodziewane warianty działania układu. Znając ich liczbę ustalić liczbę bitów przeniesienia – w tym przypadku mamy **4 stany**, do ich zakodowania potrzebne są **2 bity**. Oznaczamy je p_{i1} i p_{i2} . Stany kodujemy kolejnymi liczbami naturalnymi (można je zakodować dowolnie np. w kodzie Gray'a).

Rysujemy schemat blokowy układu, **zaznaczając stan początkowy (WAŻNE!!)** oraz schemat projektowanego jednego bloku.



Konstruujemy tablicę działania układu czyli **tablicę przejść stanów i wyjść**:

$p_{i-1,1}$	$p_{i-1,2}$	x_i	$p_{i,1}$	$p_{i,2}$	y_i
0	0	0	0	1	0
0	0	1	0	0	1
0	1	0	1	0	0
0	1	1	0	1	1
1	0	0	1	1	0
1	0	1	1	0	1
1	1	0	1	1	0
1	1	1	1	1	0

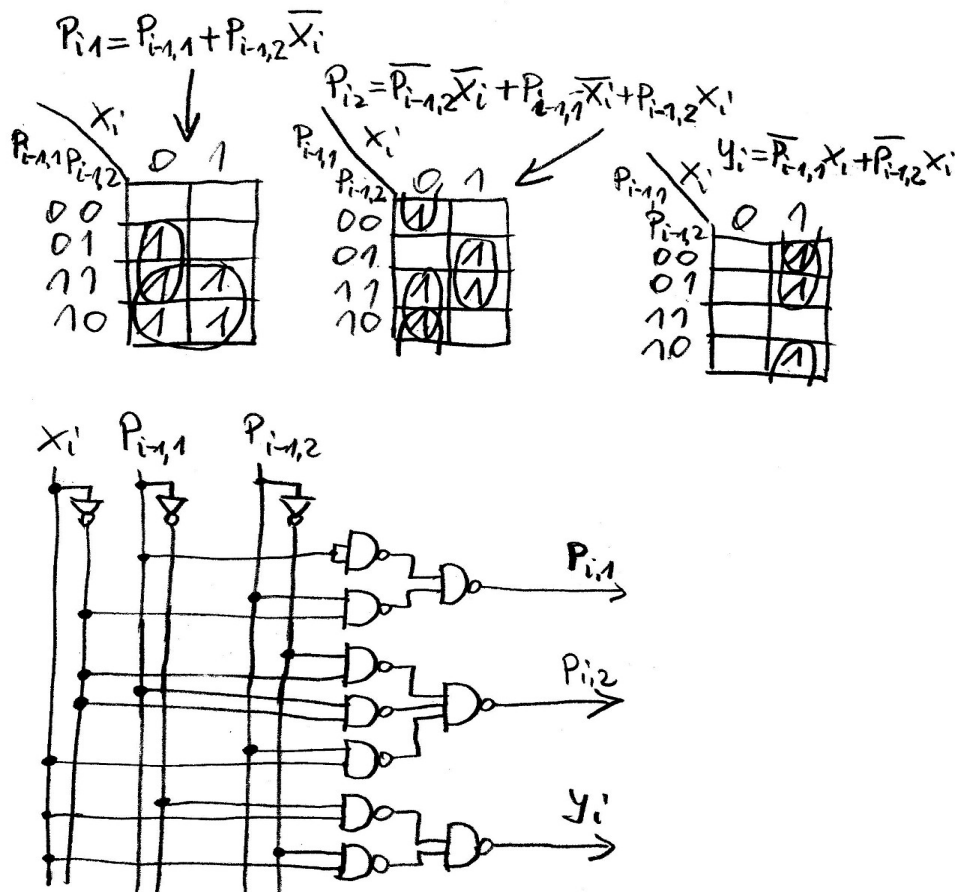
Należy zwrócić uwagę, że zmiana stanu na kolejny następuje tylko gdy na wejściu x_i pojawi się 0.

UWAGA! Gdy jesteśmy w stanie (10) i pojawi się kolejne 0, to zero to jest „przepuszczane” a nie „maskowane 0”, gdyż w dotychczas przeanalizowanej kombinacji wystąpiły dokładnie dwa zera! Dopiero w następnym bloku układu iteracyjnego będziemy w stanie (11) i wszystkie bity wejściowe będą maskowane zerami!

Konstruujemy odpowiednie tablice Karnaugh’a i wyznaczamy minimalne funkcje NPS dla:

$p_{i,1}(p_{i-1,1}, p_{i-1,2}, x_i)$, $p_{i,2}(p_{i-1,1}, p_{i-1,2}, x_i)$, $y_i(p_{i-1,1}, p_{i-1,2}, x_i)$.

Rysujemy schemat jednego bloku układu iteracyjnego.



Przykład nr VII.2

Zaprojektować i zrealizować na bramkach **NOR** iteracyjny układ kombinacyjny wykrywający w **N** elementowym słowie wejściowym sekwencję **0X0**, gdzie symbol **X** reprezentuje sygnał 0 lub 1. Układ działa w ten sposób, że jeśli poszukiwana sekwencja **pojawi się przynajmniej raz** jego **wyjście $y=1$** , natomiast w sytuacji przeciwnej **$y=0$** .

Rozwiązanie: Z treści zadania wynika, że naszym zadaniem jest wykrycie sekwencji **000** lub **010**.

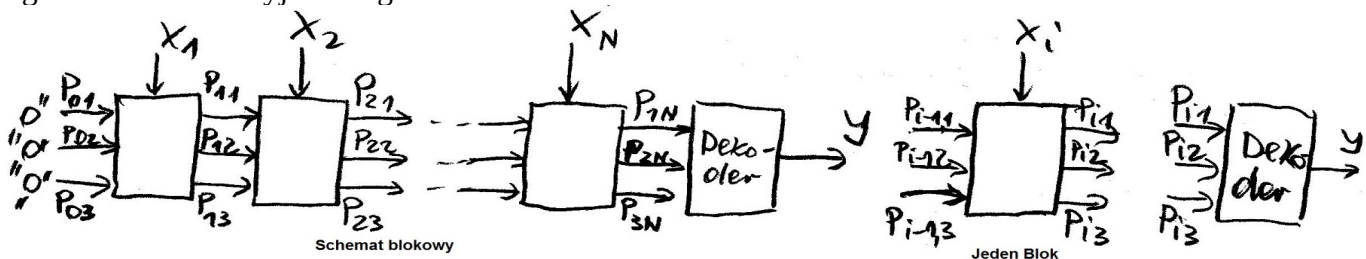
Projektowany układ będzie zbudowany z jednakowych bloków posiadających po jednym wejściu x_i . Tym razem będzie potrzebny blok wyjściowy (dekodek), gdyż wynik końcowy y , będzie wypracowywany po przeanalizowaniu całej sekwencji wejściowej – czyli po „przejrzeniu” wszystkich bloków układu iteracyjnego.

Aby narysować schemat blokowy układu oraz schematy jednego bloku iteracyjnego i bloku wyjściowego, konieczne jest ustalenie liczby stanów a tym samym liczby bitów przeniesienia. Konstruujemy poniższą tabelę:

STAN	KOD		
	p_{i1}	p_{i2}	p_{i3}
W dotychczas przeanalizowanej kombinacji nie wykryto pierwszego bitu sekwencji (0)	0	0	0
W dotychczas przeanalizowanej kombinacji wykryto pierwszy bit sekwencji (0)	0	0	1
W dotychczas przeanalizowanej kombinacji wykryto drugi bit sekwencji (00) – pierwsza wersja	0	1	0
W dotychczas przeanalizowanej kombinacji wykryto drugi bit sekwencji (01) – druga wersja	0	1	1
W dotychczas przeanalizowanej kombinacji wykryto sekwencji (000) lub (010)	1	0	0

Jak widać w tym przypadku mamy **5 stanów**, do ich zakodowania potrzebne są **3 bity**. Oznaczamy je p_{i1} , p_{i2} i p_{i3} . Stany kodujemy kolejnymi liczbami naturalnymi.

Rysujemy schemat blokowy układu, **zaznaczając stan początkowy (WAŻNE!!)** oraz schematy projektowanego jednego bloku i bloku wyjściowego.



Konstruujemy 2 tablice działania układu czyli **tablicę przejść stanów**,

$p_{i-1,1}$	$p_{i-1,2}$	$p_{i-1,3}$	x_i	$p_{i,1}$	$p_{i,2}$	$p_{i,3}$
0	0	0	0	0	0	1
0	0	0	1	0	0	0
0	0	1	0	0	1	0
0	0	1	1	0	1	1
0	1	0	0	1	0	0
0	1	0	1	0	1	1
0	1	1	0	1	0	0
0	1	1	1	0	0	0
1	0	0	0	1	0	0
1	0	0	1	1	0	0

oraz **tablicę bloku wyjściowego**:

$p_{i-1,1}$	$p_{i-1,2}$	$p_{i-1,3}$	y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1

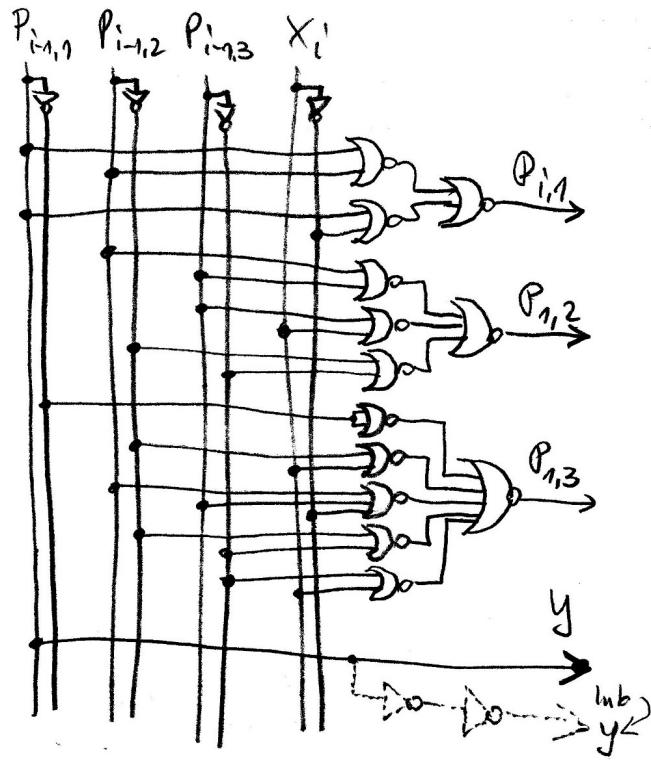
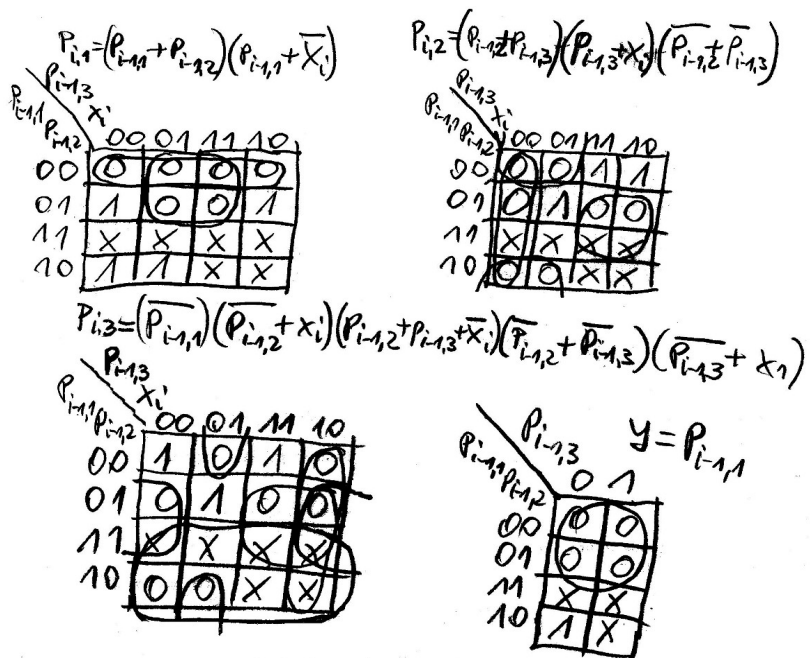
Dwie UWAGI:

- Ze stanu (010) wykryto (00) pod wpływem 1 przechodzimy do stanu (011), gdyż uzyskujemy sekwencję (001) – dwa ostatnie bity (01) „lokują” nas w stanie (011).
- Ze stanu (011) wykryto (01) pod wpływem 1 przechodzimy do stanu początkowego (000), gdyż uzyskujemy wtedy sekwencję (011) – której żaden fragment nie jest fragmentem poszukiwanego ciągu.

Konstruujemy odpowiednie tablice Karnaugh’a i wyznaczamy minimalne funkcje NPS dla:

$p_{i,1}(p_{i-1,1}, p_{i-1,2}, p_{i-1,3}, x_i)$, $p_{i,2}(p_{i-1,1}, p_{i-1,2}, p_{i-1,3}, x_i)$, $p_{i,3}(p_{i-1,1}, p_{i-1,2}, p_{i-1,3}, x_i)$, $y_i(p_{i-1,1}, p_{i-1,2}, p_{i-1,3})$.

Rysujemy schemat jednego bloku układu iteracyjnego oraz bloku wyjściowego.

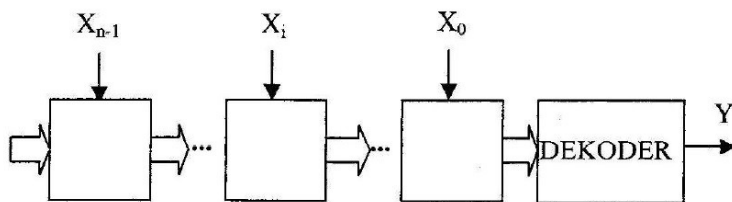


Zad. VII.1

Na wejścia układu iteracyjnego podawana jest N bitowa liczba binarna. Zaprojektować układ iteracyjny o strukturze jak na rysunku, sygnalizujący na wyjściu Y wartością $Y=1$ podzielność podanej na wejście liczby przez 3. W przeciwnym przypadku na wyjściu Y powinno pojawić się 0.

Układ zrealizować z wykorzystaniem:

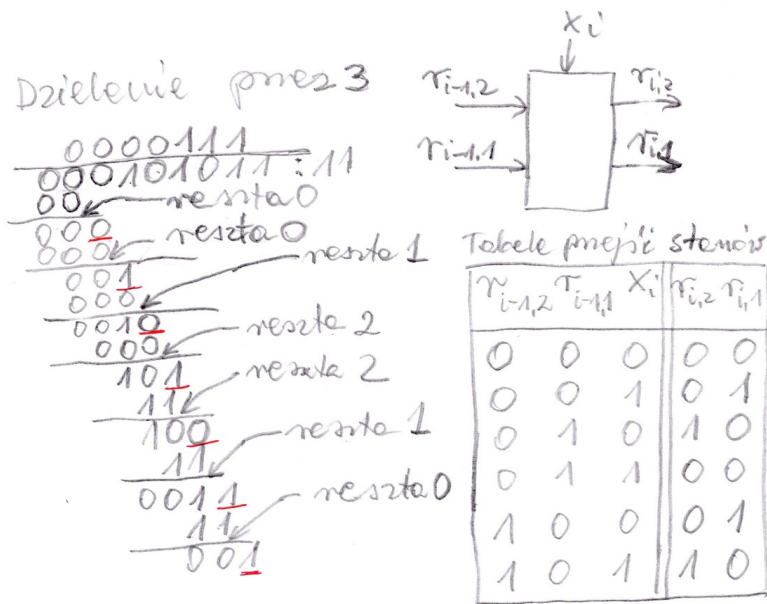
- A) bramek **NAND**,
- B) bramek **NOR**,
- C) **multiplekserów Mx 4/1** oraz bramek **NAND**.



Struktura układu iteracyjnego

Zadanie należy zrealizować samodzielnie, w sposób podany w **przykładach VII.1 i VII.2**.

Dla ułatwienia podano poniżej schemat algorytmu wyznaczania reszty z dzielenia liczby binarnej przez 3, uzyskaną za pomocą algorytmu tabelę przejść stanów oraz schemat jednego bloku układu iteracyjnego.



Bibliografia:

- M. Barski, W. Jędruch, *Układy cyfrowe, podstawy projektowania i opis w języku VHDL*, Wydawnictwo PG, Gdańsk 2007.

VIII. Projektowanie układów sekwencyjnych synchronicznych

Układy sekwencyjne są to układy cyfrowe, w których sygnały wyjściowe w danej chwili czasu zależą zarówno od bieżących sygnałów wejściowych jak i od sygnałów wejściowych z przeszłości. Są to więc **układy z pamięcią**. Do opisu ukł. sekwencyjnych wykorzystano koncepcję stanu (często stosowaną w innych dziedzinach nauki). **Przez stan rozumiemy taki zespół wielkości opisujących wewnątrz układu w danej chwili czasu, że znajomość jego i aktualnego sygnału wejściowego pozwala na wyznaczenie aktualnego sygnału wyjściowego oraz następnego stanu.** W aktualnym stanie zawiera się dotychczasowa historia pracy układu.

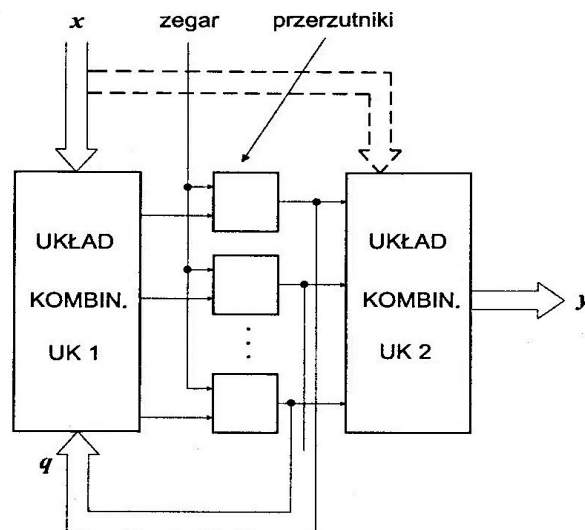
Wyróżnia się dwa modele układów sekwencyjnych:

$$\text{Układ Moore'a} \quad \begin{cases} \bar{q}_{k+1} = \bar{\delta}(\bar{q}_k, \bar{x}_k) \\ \bar{y}_k = \bar{\lambda}(\bar{q}_k) \end{cases} \quad \text{i układ Mealy'ego} \quad \begin{cases} \bar{q}_{k+1} = \bar{\delta}(\bar{q}_k, \bar{x}_k) \\ \bar{y}_k = \bar{\lambda}(\bar{q}_k, \bar{x}_k) \end{cases}$$

gdzie: $\bar{q}_i, \bar{x}_i, \bar{y}_i$, to **wektory** odpowiednio stanu, wejścia i wyjścia w chwili i , $\bar{\delta}$ jest **wektorem funkcji przejścia** (zmiany rektora stanu) a $\bar{\lambda}$ to **wektor funkcji wyjścia**. Do **opisu układów sekwencyjnych stosujemy tablice przejść/wyjść (przejść stanów i wyjść układu) lub równoważnie graf**. W porównaniu do układów kombinacyjnych układy sekwencyjne zawierają **dodatkowe elementy**. Są to **układy pamiętające zwane przerzutnikami**. Układy sekwencyjne synchroniczne tym różnią się od układów sekwencyjnych asynchronicznych, że posiadają **dodatkowe wejście zwane zegarowym** służące do sterowania zapisem informacji w przerzutnikach. Stąd ich projektowanie jest prostsze.

Schemat blokowy synchronicznego układu sekwencyjnego zawiera rysunek poniżej. Składa się on z **Układu kombinacyjnego UK1**, warstwy **przerzutników** oraz **Układu Kombinacyjnego UK2**. Układ zawiera sprzężenie zwrotne od wyjścia przerzutników do wejścia UK1. W układzie Mealy'ego do UK2 doprowadzany jest również sygnał wejściowy.

Liczba zastosowanych przerzutników zależy od **liczby bitów potrzebnych do zakodowania wszystkich możliwych stanów** układu (np. do zakodowania 4 stanów wystarczą 2 bity -> 2 przerzutniki a do zakodowania 5 stanów potrzebne będą już 3 bity -> 3 przerzutniki). W trakcie pracy układu sekwencyjnego zmiany sygnałów wejściowych będą powodowały „wędrowanie” po grafie wektora stanu. **Zadaniem UK1 jest takie wysterowanie przerzutników, aby pod wpływem sygnałów wejściowych układ zmieniał stan zgodnie grafem projektowanego układu.** UK1 realizuje funkcję $\bar{\delta}(\bar{q}_k, \bar{x}_k)$. UK2 wyznacza w zależności od tego czy zastosowaliśmy model Moore'a czy Mealy'ego wektor funkcji wyjściowych odpowiednio $\bar{\lambda}(\bar{q}_k)$. lub $\bar{\lambda}(\bar{q}_k, \bar{x}_k)$.



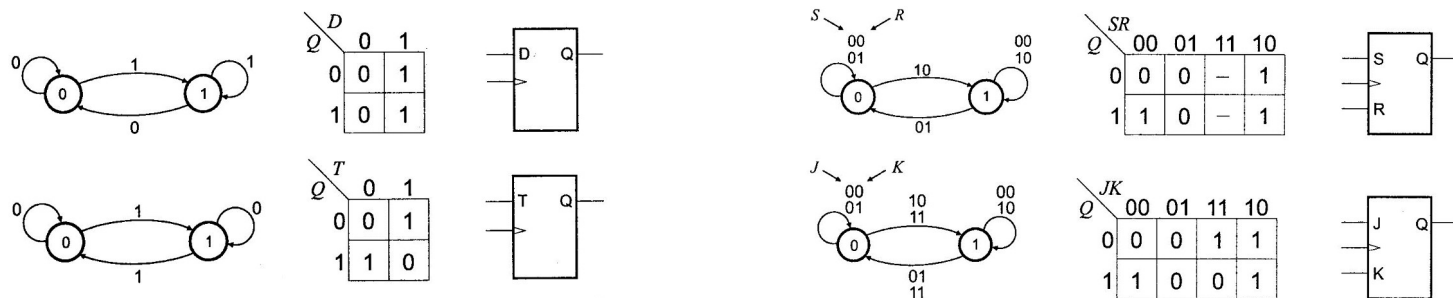
UWAGA! Można pokazać, że **układ sekwencyjny jest „wirtualną” wersją iteracyjnego układu kombinacyjnego**, w którym UK1 odpowiada jednemu blokowi układu iteracyjnego (w przypadku zastosowania przerzutników typu D, o których za chwilę, jest to dokładnie taki sam układ – to samo NPS lub NPI).

UK2 jest tożsamy z blokiem wyjściowym układu iteracyjnego (dekoderem).

Kolejne stany przenoszone pomiędzy blokami układu iteracyjnego są w układzie sekwencyjnym zapamiętywane w warstwie przerzutników - zadanie realizowane przez układ iteracyjny „w przestrzeni” (przez wiele bloków) jest w układach sekwencyjnych realizowane „w czasie” przez „jeden blok” i warstwę przerzutników.

Wnioski: układy sekwencyjne są wolniejsze od iteracyjnych układów kombinacyjnych mają za to prostszą budowę.

Typy przerzutników:



Na potrzeby syntezy układów stosuje się tzw. **tablice wzbudzeń przerzutników (sterowania przerzutnikami)**:

Q_k	Q_{k+1}	D	T	SR	JK
0	0	0	0	0 ×	0 ×
0	1	1	1	1 0	1 ×
1	0	0	1	0 1	× 1
1	1	1	0	× 0	× 0

Etapy projektowania układu sekwencyjnego synchronicznego:

1. Opis słowny działania układu.
2. Synteza abstrakcyjna czyli utworzenie opisu formalnego w postaci grafu lub tablicy przejść/wyjść.
3. Minimalizacja liczby stanów – zastąpienie danego układu sekwencyjnego innym o mniejszej liczbie stanów, identycznie reagującym na interesujący nas zbiór sygnałów wejściowych. (etap minimalizacji na tym kursie pomijamy!).
4. Kodowanie, czyli przyporządkowanie liczb binarnych poszczególnym stanom w grafie lub tablicy przejść.
5. Synteza kombinacyjna **UK1** i **UK2**.

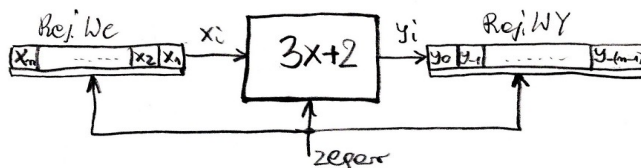
Przykład nr VIII.1

Zaprojektować układ **sekwencyjny synchroniczny** wykonujący operację arytmetyczną $Y=3X+2$. Układ ma posiadać jedno-bitowe wejście i jedno-bitowe wyjście. N-bitowa liczba X ma być podawana na wejście układu za pomocą rejestru szeregowego, bit po bicie, od najmniej znaczącej pozycji. Kolejne bity liczby Y (wyniku operacji arytmetycznej) mają być zapisywane bit po bicie w szeregowym rejestrze wyjściowym. Końcowy wynik operacji arytmetycznej będzie odczytany (równolegle) z rejestru wyjściowego po N impulsach zegarowych.

Układ zrealizować następująco:

- **najbardziej znaczący bit kodu stanu** na przerzutniku **D** oraz bramkach **NAND**,
- **pozostałe bity kodu stanu** na przerzutnikach **JK** i bramkach **NOR**,
- **układ wyjściowy** z wykorzystaniem **multipleksera $M \times 4 \times 1$** , przy czym na wejścia adresowe $A_1 A_0$ należy podać dwa najbardziej znaczące bity kodu stanu a funkcje pozostałych bitów kodu stanu (jeśli będą) oraz bitu sygnału wejściowego x_i na wejścia informacyjne multipleksera.

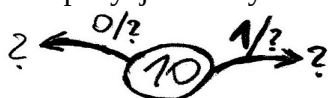
Schemat blokowy układu przedstawia rysunek.



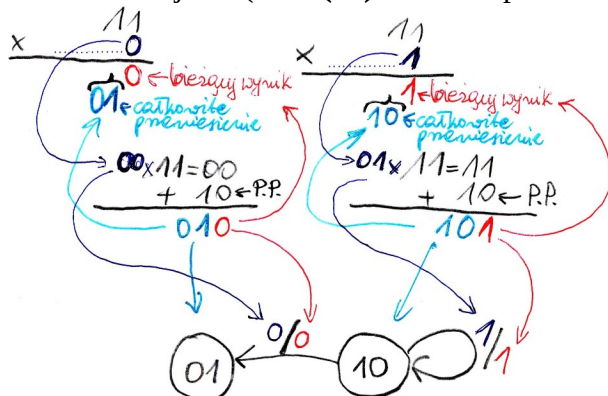
Rozwiązanie: Operacja arytmetyczna $Y=3X+2$ zawiera mnożenie. Układy mnożące (również dzielące) najprościej jest zrealizować jako **układ Mealy’ego**, w którym rolę **stanu pełni całkowite przeniesienie do następnych pozycji, obliczone na aktualnej pozycji binarnej**. Liczba 3 będzie mnożona przez kolejno pojawiające się bity x_i liczby X . Do wyniku mnożenia będzie **dobawane całkowite przeniesienie z poprzedniej pozycji binarnej**. **Najmniej znaczący bit** uzyskanego wyniku będzie bieżącym **bitem wyjściowym y_i** a **pozostałe bity wyniku** (starsze) będą stanowiły całkowite przeniesienie do następnych pozycji – czyli

wyznaczą następny stan układu. W tym zadaniu **projekt rozpoczniemy od narysowania grafu**. W ten sposób **wyznamy całkowitą liczbę stanów** oraz **tablicę przejść/wyjść** układu.

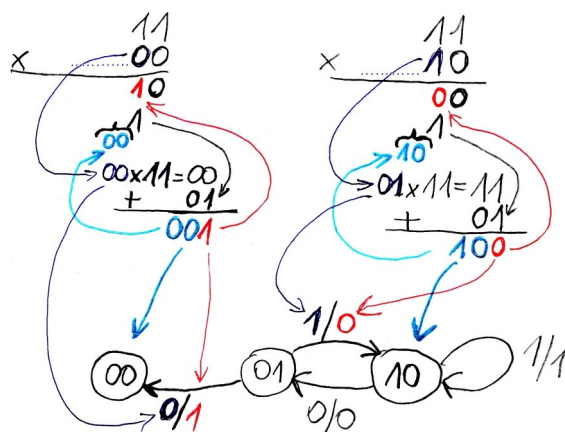
Ponieważ w wyrażeniu arytmetycznym do wyniku mnożenia $3X$ dodawana jest liczba 2, stanem początkowym, czyli „całkowitym przeniesieniem z poprzednich pozycji binarnych” będzie $2=(10)_2$.



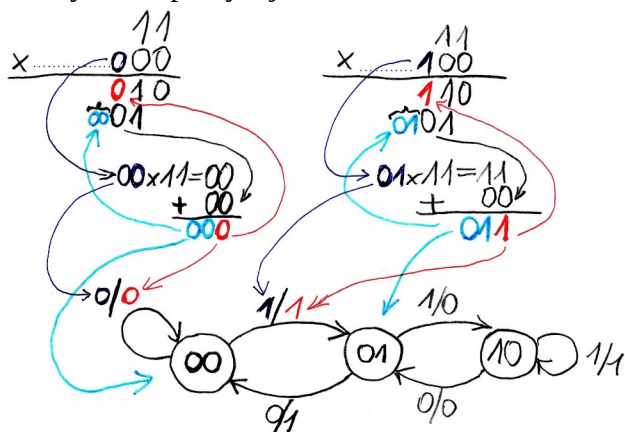
Projektowany układ ma jedno-bitowe wejście, stąd należy określić jakie przejścia uzyskamy, z jakim sygnałem wyjściowym, gdy x_i będzie równe odpowiednio 0 lub 1. Oznacza to, że ze stanu początkowego (również każdego innego) „muszą wychodzić” dwie strzałki. Na poniższym rysunku pokazano jakie stany i wyjścia zostaną wygenerowane dla odpowiednio 0 lub 1 na wejściu (**P.P.**=(10)₂ oznacza przeniesienie początkowe).



Jak można zauważyć dla $x_i = 0$ wygenerował się nowy stan odpowiadający przeniesieniu (01)₂ a dla dla $x_i = 1$ powróciliśmy do stanu (10)₂. Na czerwono zaznaczono bieżący sygnał wyjściowy y_i . Na kolejnym rysunku pokazano podobne zabiegi przeprowadzone dla stanu (01)₂. Odpowiadają one dalszemu rozwiązywaniu lewego równania z rysunku powyżej.



Jak można zauważyć dla $x_i = 0$ wygenerował się nowy stan odpowiadający przeniesieniu (00)₂ a dla dla $x_i = 1$ powróciliśmy do stanu (10)₂. Na czerwono zaznaczono podobnie jak poprzednio bieżący sygnał wyjściowy y_i . Na kolejnym rysunku pokazano podobne zabiegi przeprowadzone dla stanu (00)₂. Odpowiadają one dalszemu rozwiązywaniu lewego równania z rysunku powyżej.



Ostatecznie uzyskujemy graf o trzech już zakodowanych stanach: $(00)_2$, $(01)_2$, $(10)_2$ (etap kodowania stanów mogliśmy tym razem pominąć). Dalsze etapy projektu zawiera poniższy rysunek.

- Na podstawie grafu tworzymy tablicę przejść/wyjść układu (jest to tablica modelu Mealy'ego).

Ponieważ stanów jest mało od razu tworzymy tablicą Karnaugh'a przejść/wyjść – zawiera ona wszystkie przejścia i wyjścia nawet te nieokreślone dla nieużywanego kodu stanu $(11)_2$.

- Pomijamy etap minimalizacji liczby stanów (można wykazać, że ten graf jest minimalny).

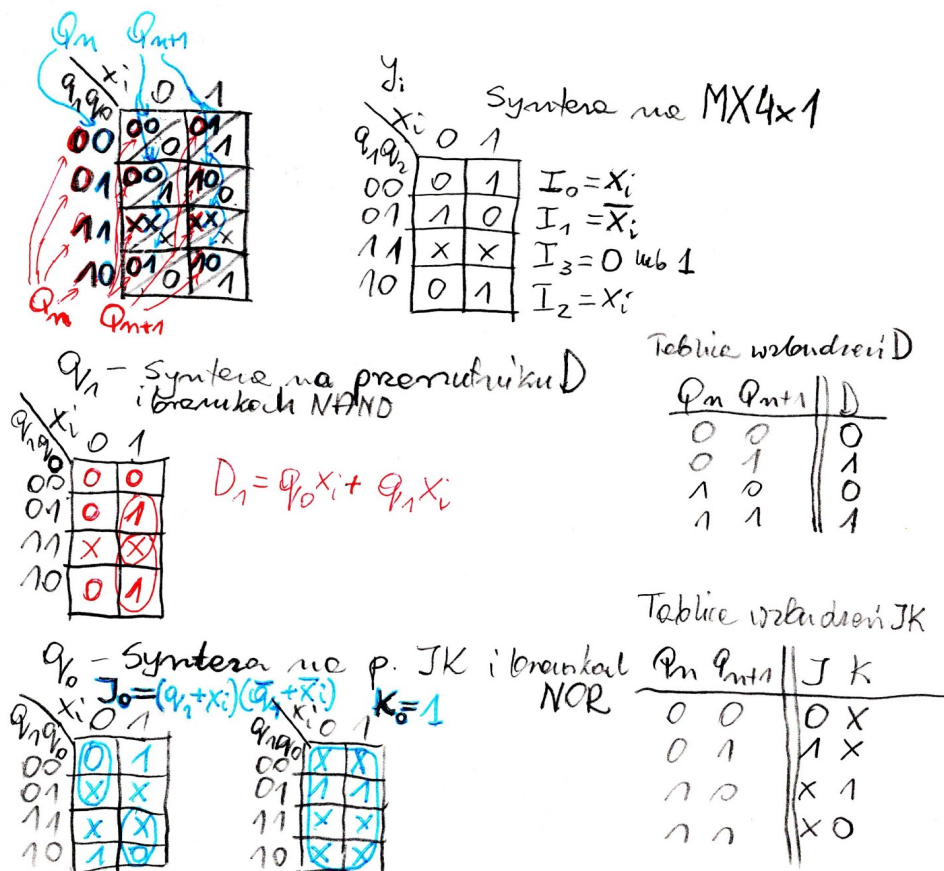
- Na podstawie tablicy przejść/wyjść tworzymy odpowiednio ukształtowaną tablicę Karnaugh'a układu wyjściowego (**UK2**), którą będziemy syntezować na $M \times 4 \times 1$. Dane do tablicy odczytujemy poniżej kreski dzielącej komórki na pół. Ponieważ stan zapisywany jest na 2 bitach na wejścia informacyjne multiplexera pošlemy funkcje tylko zmiennej x_i . Dla argumentów $(11)_2$ funkcja I_3 może przyjąć dowolną wartość, ostatecznie określimy ją w trakcie syntezy tak by była łatwa do realizacji.

- Synteza **UK1** przebiega następująco. W tablicy przejść na czerwono zaznaczono bardziej znaczący bit stanu q_1 , a na błado-niebiesko mniej znaczący bit stanu q_0 . Na boku tablicy bity te posiadają wartość w chwili Q_n a wewnątrz w chwili Q_{n+1} . Bit stanu q_1 zostanie zrealizowany na przerzutniku **D**. Tablice funkcji sterującej przerzutnikiem utworzymy wykorzystując **Tablicę Wzbudzeń** przerzutnika **D**. Przykładowo komórkę tablicy o współrzędnych: $(q_1 q_0) = (00)_2$ i $x_i = 0$ (lewy górny róg) wyznaczmy obserwując, jak zmienia się bit q_1 z wartości na boku do wartości wewnątrz tablicy: widzimy zmianę $0 \rightarrow 0$. Patrząc do tablicy wzbudzeń przerzutnika **D** widzimy, że przejściu $0 \rightarrow 0$ odpowiada wartość $D=0$. Taką też wartość wpisujemy do tablicy funkcji przerzutnika **D** (lewy górny róg). Z pozostałymi komórkami tablicy postępujemy podobnie.

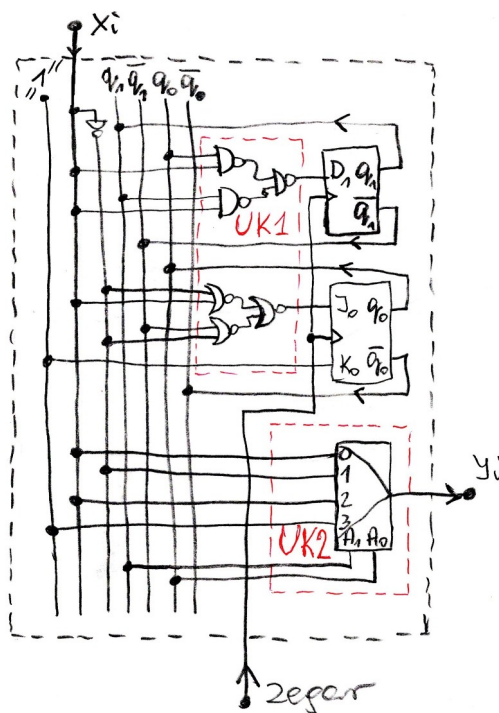
Jak można zauważyć stosując przerzutnik **D** otrzymujemy tablicę identyczną z tablicy przejść, która odpowiada bitowi stanu q_1 , stąd **wniosek: synteza na przerzutnikach D jest prosta - zakodowana tablica Karnaugh'a przejść jest już tablicą funkcji sterujących przerzutnikami D**.

Tablice funkcji sterującej przerzutnikiem **D** minimalizujemy względem **1 (NAND)**.

Bit stanu q_0 zostanie zrealizowany na przerzutniku **JK**. Tablice **dwóch funkcji sterujących** wejściami przerzutnika **J** i **K** utworzymy w sposób opisany powyżej obserwując zmiany bitu q_0 i wykorzystując **Tablicę Wzbudzeń** przerzutnika **JK**. Najwygodniej jest wypełniać obie tablice naraz odczytując odpowiednie dwie wartości z Tablicę Wzbudzeń. Tablice minimalizujemy względem **0 (NOR)**.



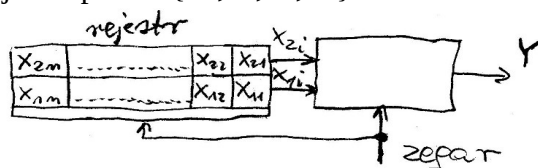
Schemat układu zamieszczono na poniższym rysunku. Ponieważ $K_0=1$ przyjęto, że wejście I_3 multiplexera w **UK2** też jest równe 1.



Przykład nr VIII.2

Zaprojektować i zrealizować na przerzutnikach **SR** i bramkach **NAND** układy sekwencyjny synchroniczny wykrywający w wejściowym ciągu cyfr czwórkowych sekwencję **1312**. Układ działa w ten sposób, że jeśli poszukiwana sekwencja **pojawi się przynajmniej raz** jego **wyjście $y=1$** , natomiast w sytuacji przeciwnej **$y=0$** .

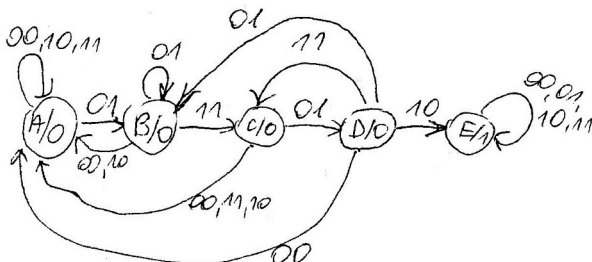
Rozwiązanie: Z treści zadania wynika, że projektowany układ ma 2 bitowe wejście i 1 bitowe wyjście. W kodzie czwórkowym poszukiwana sekwencja ma postać **{01,11,01,10}**. Schemat blokowy układu przedstawia rysunek.



Aby skonstruować graf układu w tym zadaniu konieczne jest ustalenie liczby stanów, a tym samym liczby bitów potrzebnych do ich zakodowania (- liczby potrzebnych przerzutników). Konstruujemy poniższą tabelę:

STAN	Oznaczenia stanów
W ciągu wejściowym nie wykryto początku sekwencji	A
W ciągu wejściowym wykryto pierwszą cyfrę czwórkową sekwencji {01}	B
W ciągu wejściowym wykryto pierwszą i drugą cyfrę czwórkową sekwencji {01,11}	C
W ciągu wejściowym wykryto trzy pierwsze cyfry czwórkowe sekwencji {01,11,01}	D
W ciągu wejściowym wykryto sekwencję {01,11,01,10}	E

Układ ma 5 stanów. Znając liczbę stanów oraz budowę poszukiwanej sekwencji (następstwo cyfr) skonstruowano graf układu. Zastosowano **model Moore'a**, typowy dla zadań dotyczących wykrywania sekwencji.



Poszczególne etapy projektowania układu zawiera poniższy rysunek. Tablicę Karnaugh'a dekodera zrealizowano wpisując do odpowiednich komórek wartości z kolumny tablicy przejść/wyjść dotyczącej **Y**. Jak można zauważyć (co jest zgodne z omawianą wcześniej teorią), **wyjście układu Moore'a zależy wyłącznie od bitów stanu**. Należy również zwrócić uwagę na opis lewego boku zakodowanej tablicy przejść/wyjść – nie jest to kod Graya, co w tym

miejsu nie jest błędem oraz na fakt „obrócenia” o 90° zakodowanej tablicy Karnaugh’a przejść/wyjść w stosunku do zakodowanej tablicy przejść/wyjść - wiersze stały się kolumnami.

Tablica przejść/wyjść

$x_2 x_1$	00	01	11	10	Y
A	A	B	A	A	0
B	A	B	C	A	0
C	A	D	A	A	0
D	A	B	C	E	0
E	E	E	E	E	1

Kodowanie stanów

Q	$Q_3 Q_2 Q_1$
A	000
B	001
C	010
D	011
E	100

Zakodowana tablica przejść/wyjść

$Q_3 Q_2 Q_1$	00	01	11	10	Y
000	000	001	000	000	0
001	000	001	010	000	0
010	000	011	000	000	0
011	000	001	010	100	0
100	100	100	100	100	1

Tablica Karnaugh'a dekodera

$Q_3 Q_2 Q_1$	00	01	11	10
0	0	0	0	0
1	1	X	X	X

$Y = Q_3$

Zakodowana tablica Karnaugh'a przejść/wyjść

$Q_3 Q_2 Q_1$	000	001	011	010	110	111	101	100
00	000	000	000	000	X	X	X	100
01	001	001	001	011	X	X	X	100
11	000	010	010	000	X	X	X	100
10	000	000	100	000	X	X	X	100

Tablica wzbudzeń przerzutnika SR

Q_{n+1}	Q_n	S R
0	0	0 X
0	1	1 0
1	0	0 1
1	1	X 0

Tablice Karnaugh'a funkcji sterujących przerzutnikami $S_3 R_3, S_2 R_2, S_1 R_1$

$Q_3 Q_2 Q_1$	000	001	011	010	110	111	101	100
00	0	0	0	0	X	X	X	X
01	0	0	0	0	X	X	X	X
11	0	0	0	0	X	X	X	X
10	0	0	1	0	X	X	X	X

$S_3 = x_2 \bar{x}_1 \bar{Q}_1$

$Q_3 Q_2 Q_1$	000	001	011	010	110	111	101	100
00	0	0	0	0	X	X	X	0
01	0	0	0	0	X	X	X	0
11	0	1	X	0	X	X	X	0
10	0	0	0	0	X	X	X	0

$S_2 = x_2 x_1 Q_1$

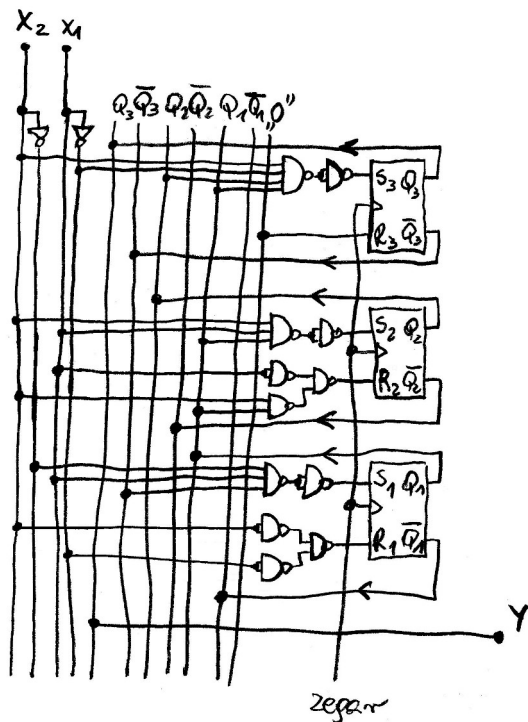
$Q_3 Q_2 Q_1$	000	001	011	010	110	111	101	100
00	0	0	0	0	X	X	X	0
01	0	0	0	0	X	X	X	0
11	0	1	X	0	X	X	X	0
10	0	0	0	0	X	X	X	0

$S_1 = \bar{x}_2 x_1 \bar{Q}_3$

$Q_3 Q_2 Q_1$	000	001	011	010	110	111	101	100
00	0	1	1	X	X	X	X	X
01	0	0	0	0	X	X	X	X
11	0	1	1	X	X	X	X	X
10	0	1	1	X	X	X	X	X

$R_1 = x_2 + \bar{x}_1$

Schemat układu zamieszczono na poniższym rysunku.



Materiały dydaktyczne do przedmiotu
Układy Cyfrowe – Projekt (informatyka sem. II)
dr inż. Krzysztof Cisowski
Katedra Systemów Automatyki, Wydz. EiT PG

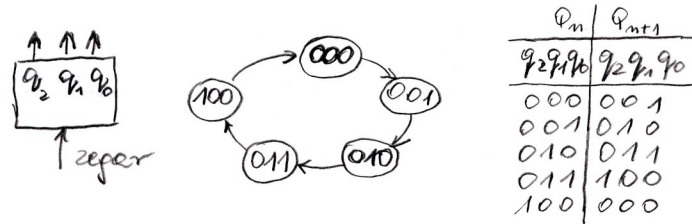
VIII. Projektowanie układów sekwencyjnych synchronicznych c.d.

Przykładowe zadania.

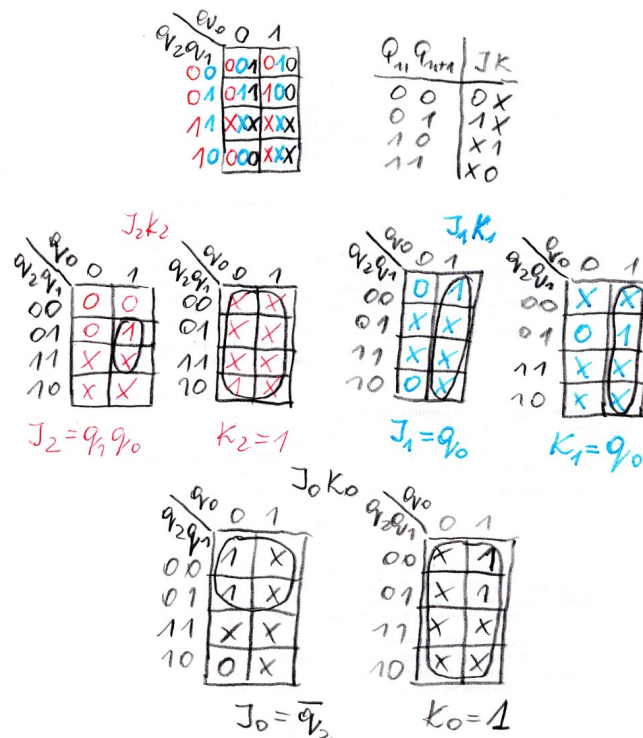
Zad. VIII.1

Zaprojektować i zrealizować na przerzutnikach JK i bramkach NAND licznik binarny **modulo 5** liczący w **kodzie binarnym w przód**.

Rozwiązanie: Schemat blokowy, graf oraz **tablica przejść** projektowanego układu wyglądają następująco (etap kodowania nie był potrzebny):



Jak można zauważyć układ nie posiada wejścia a wyjściem są wartości odczytywane z wyjść przerzutników. Na podstawie tablicy przejść utworzono **tablicę Karnaugh'a przejść stanów** - **kolorami wyróżniono** poszczególne bity. Korzystając z **tablicy wzbudzeń** przerzutnika JK skonstruowano „kolorowe” tablice dla poszczególnych przerzutników JK realizujących bity stanów q_2, q_1 i q_0 .

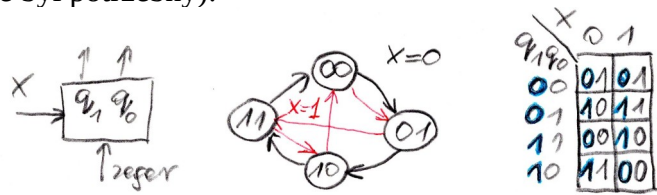


W ramach ćwiczenia należy narysować schemat zaprojektowanego układu.

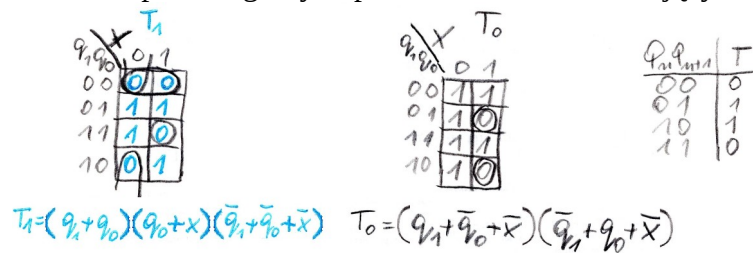
Zad. VIII.2

Zaprojektować i zrealizować na przerzutnikach **T** i bramkach **NOR** licznik binarny **modulo 4** z wejściem sterującym **x**, liczący **w przód**. Gdy **x=0** licznik liczy w **kodzie binarnym** a gdy **x=1** w **kodzie Gray’a**.

Rozwiązanie: Schemat blokowy, graf oraz **tablica Karnaugh’a przejść** projektowanego układu wyglądają następująco (etap kodowania nie był potrzebny):



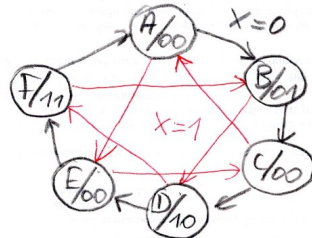
Jak można zauważyć wyjściem układu są wartości odczytywane z wyjść przerzutników. W **tablicy Karnaugh’a przejść stanów kolorami wyróżniono** poszczególne bity stanu. Korzystając z **tablicy wzbudzeń** przerzutnika **T** skonstruowano „kolorowe” tablice dla poszczególnych przerzutników **T** realizujących bity stanów **q1** i **q0**.



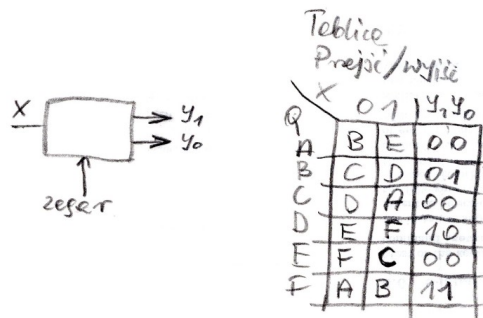
W ramach ćwiczenia należy narysować schemat zaprojektowanego układu.

Zad. VIII.3

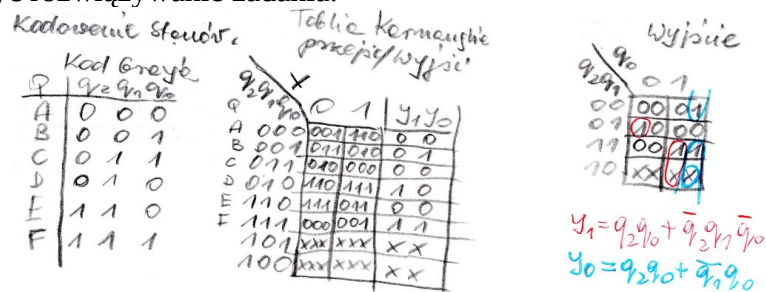
Zaprojektować i zrealizować na przerzutnikach **SR** i bramkach **NAND** licznik działający zgodnie z podanym grafem.



Rozwiązanie: Schemat blokowy, oraz **tablica przejść/wyjść** projektowanego układu umieszczono poniżej. Jest to **układ Moore’a** z **jedno-bitowym wejściem** i **dwu-bitowym wyjściem**.



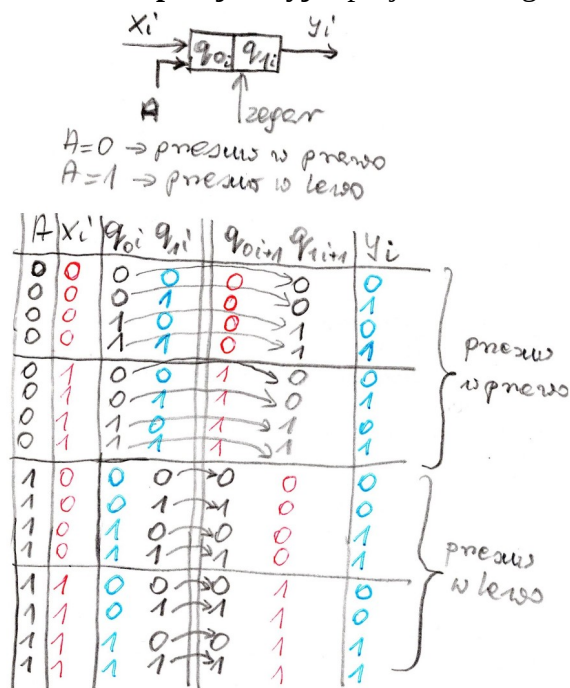
Zastosowano **kodowanie stanów w kodzie Gray’a**, Poniżej pokazano **zakodowaną tablice Karnaugh’a przejść/wyjść** oraz **wyznaczono funkcje wyjścia bitów y1 i y0** (zależne tylko od stanu – gdyż jest to układ **Moore’a**). Należy dokończyć rozwiązywanie zadania.



Zad. VIII.4

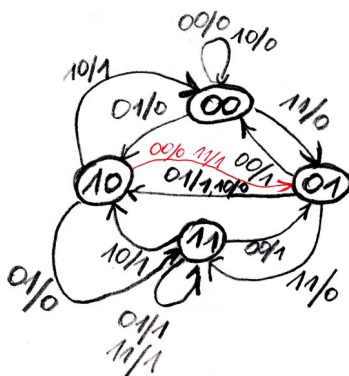
Zaprojektować i zrealizować na przerzutnikach **D** i bramkach **NAND** **dwu (2) bitowy** rejestr **SISO** (*Serial Input Serial Output*) z wejściem sterującym **A**. jeśli **A=0** to dane przesuwane są w prawo a gdy **A=1** w lewo.

Rozwiązanie: Schemat blokowy, oraz **tablica przejść/wyjść** projektowanego układu umieszczono poniżej.



Projektowany układ zostanie zrealizowany z wykorzystaniem **modelu Mealy'ego**. Układ działa w ten sposób, że gdy **dane przesuwane** są **w prawo**, to w danej chwili czasu **i** wyjście **$y_i = q_{1(i)}$** a w kolejnej chwili **i+1** otrzymujemy **$q_{1(i+1)} = q_{0(i)}$** , **$q_{0(i+1)} = x_i$** (zaznaczono to kolorami i strzałkami). Natomiast, gdy **dane przesuwane** są **w lewo**, to w danej chwili czasu **i** wyjście **$y_i = q_{0(i)}$** a w kolejnej chwili czasu **i+1** mamy **$q_{0(i+1)} = q_{1(i)}$** , **$q_{1(i+1)} = x_i$** . W oparciu o tablicę przejść/wyjść utworzono **tablicę Karnaugh'a przejść/wyjść** oraz **graf układu**. Należy dokończyć rozwiązywanie zadania.

$q_{0,i} q_{1,i} \backslash x_i$	00	01	11	10
00	00/0	00/1	01/1	01/0
01	10/0	10/1	11/1	11/0
11	01/0	11/0	11/1	01/1
10	00/0	10/0	10/1	00/1



Zad. VIII.5

Zaprojektować i zrealizować na przerzutnikach **D** i bramkach **NAND** **trzy (3) bitowy** rejestr **SISO** (*Serial Input Serial Output*) z **przesuwaniem** danych **w prawo**.

Zadanie należy zrealizować samodzielnie w wersji Mealy'ego rozpoczynając od budowy tablicy przejść/wyjść!